



《模式识别》

第二章 特征提取与表示

马锦华

<https://cse.sysu.edu.cn/teacher/MaJinhua>

SUN YAT-SEN University



声明：该PPT只供非商业使用，也不可视为任何出版物。由于历史原因，许多图片尚没有标注出处，如果你知道图片的出处，欢迎告诉我们 majh8@mail.sysu.edu.cn.



课程目录（暂定）

❑	第一章	课程简介与预备知识	6学时
❑	第二章	特征提取与表示	6学时
❑	第三章	主成分分析	3学时
❑	第四章	归一化、判别分析、人脸识别	3学时
❑	第五章	EM算法与聚类	3学时
❑	第六章	贝叶斯决策理论	3学时
❑	第七章	线性分类器与感知机	3学时
❑	第八章	支持向量机	3学时
❑	第九章	神经网络、正则项和优化方法	3学时
❑	第十章	卷积神经网络及经典框架	3学时
❑	第十一章	循环神经网络	3学时
❑	第十二章	Transformer	3学时
❑	第十三章	自监督与半监督学习	3学时
❑	第十四章	开放世界模式识别	6学时



预备知识：图像处理基础

Images and Transformations

Types of Images

Binary



Grayscale



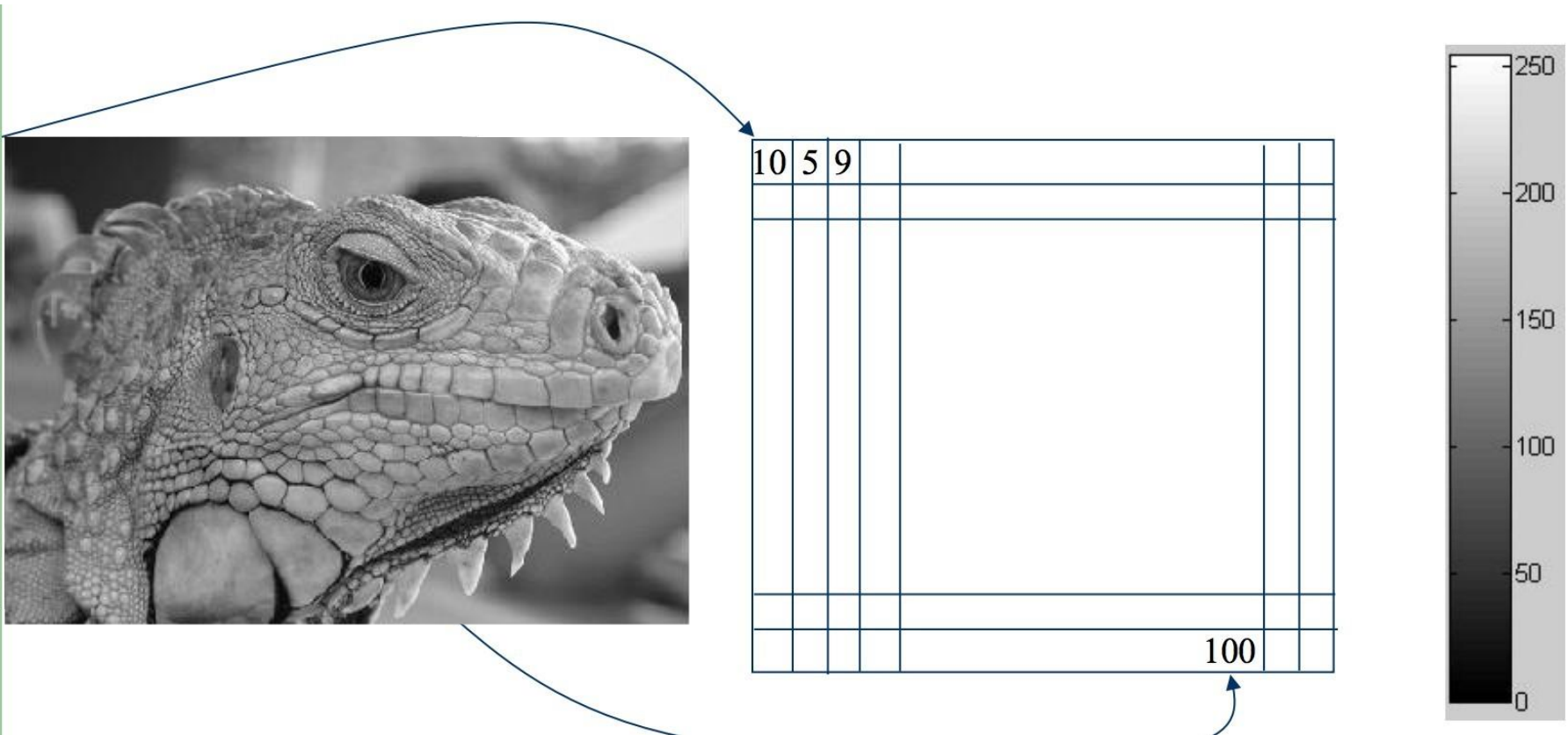
Color





6

Grayscale image representation



Slide credit: Ulas
Bagci

Color image representation



Phil Noble / AP



B channel



Phil Noble / AP

G channel

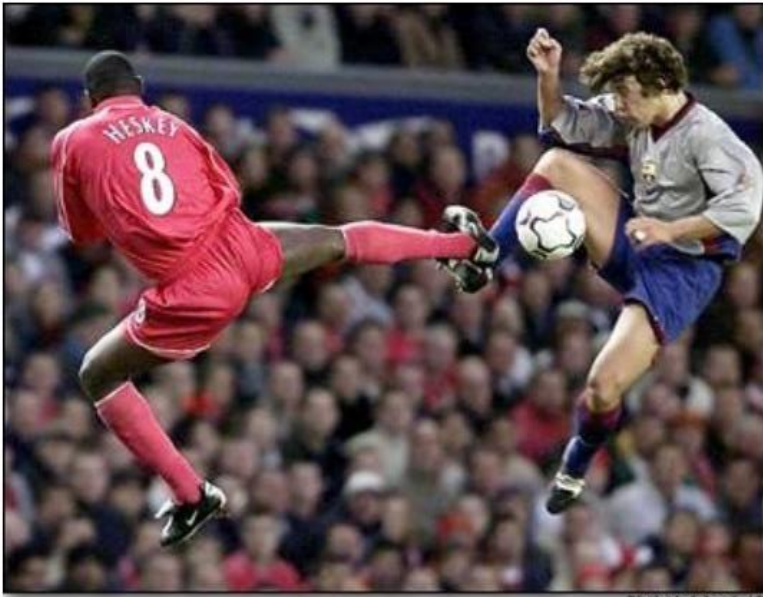


Phil Noble / AP

R channel

Slide credit: Ulas
Bagci

Color image - one channel



Phil Noble / AP

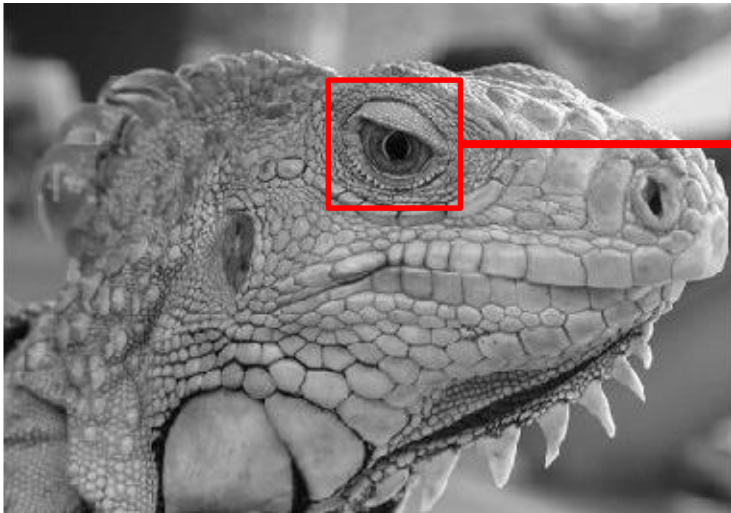


Phil Noble / AP

R channel

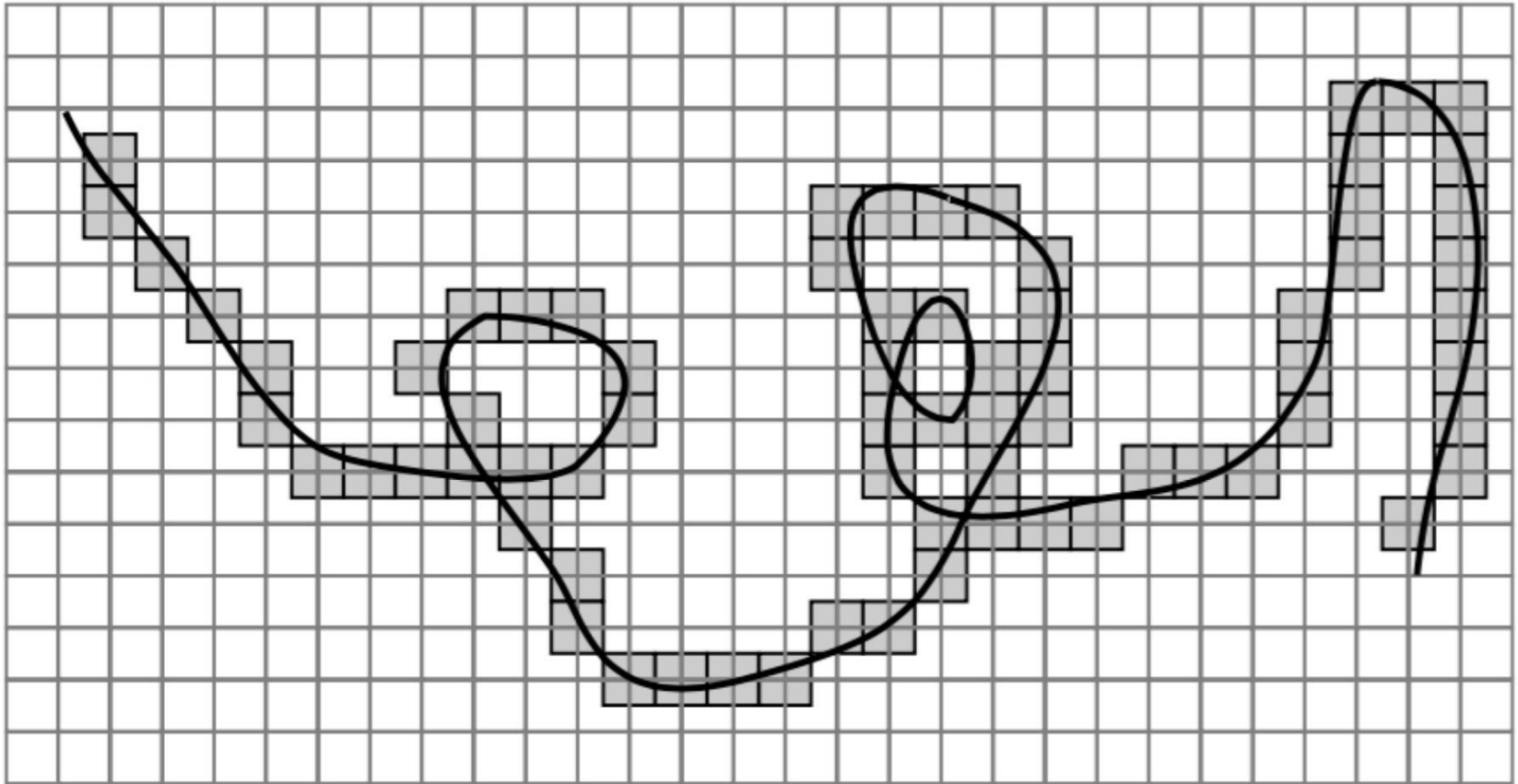


Digital Images are sampled



- What happens when we zoom into the images we capture?

Errors due to Sampling



Slide credit: Ulas
Bagci

Resolution

*is a **sampling** parameter, defined in dots per inch (DPI) or equivalent measures of spatial pixel density*



Slide credit: Ulas
Bagci

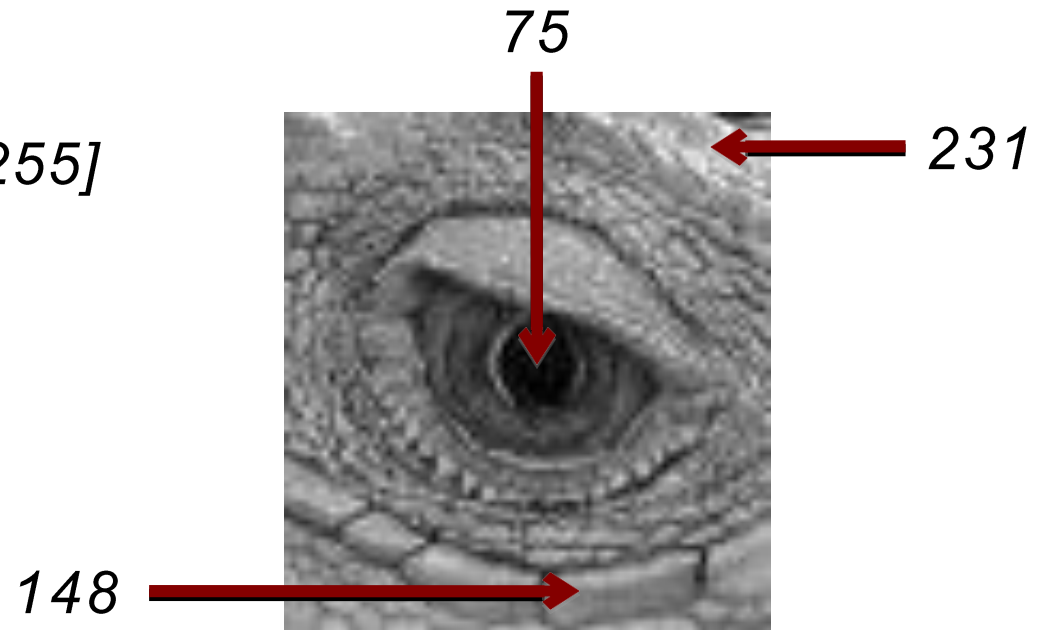
Images are Sampled and Quantized

- An image contains discrete number of pixels

–Pixel value:

- “*grayscale*”

(or “*intensity*”): $[0, 255]$



Images are Sampled and Quantized

- An image contains discrete number of pixels

–Pixel value:

- “grayscale”

(or “intensity”): $[0, 255]$

- “color”

–RGB: $[R, G, B]$

$[90, 0, 53]$



$[249, 215, 203]$

$[213, 60, 67]$

With this loss of information (from sampling and quantization),

Can we still use images for useful tasks?



Summary

- *Image types (binary, grayscale, color)*
- *Images are sampled and quantized*

2D transformations

- *Transformation Matrices*
- *Homogeneous coordinates*
- *Translation*
- *Scaling*
- *Rotation*

Transformation Matrices

- *Matrices can be used to transform vectors in useful ways, through multiplication: $p' = A p$*
- *Simplest is scaling:*

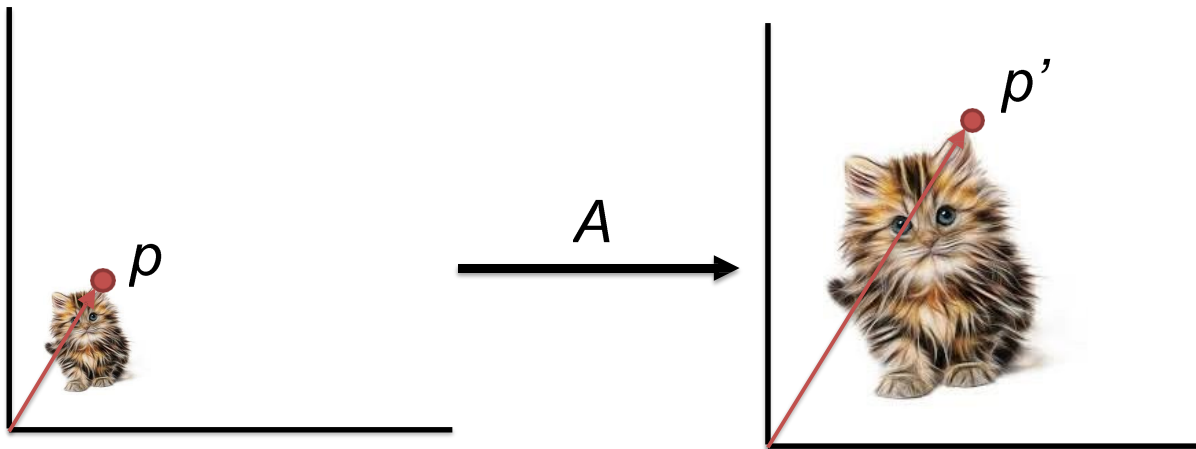
$$\begin{array}{c} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \\ A \end{array} \times \begin{array}{c} \begin{bmatrix} x \\ y \end{bmatrix} \\ p \end{array} = \begin{array}{c} \begin{bmatrix} s_x x \\ s_y y \end{bmatrix} \\ p' \end{array}$$

(Verify to yourself that the matrix multiplication works out this way)

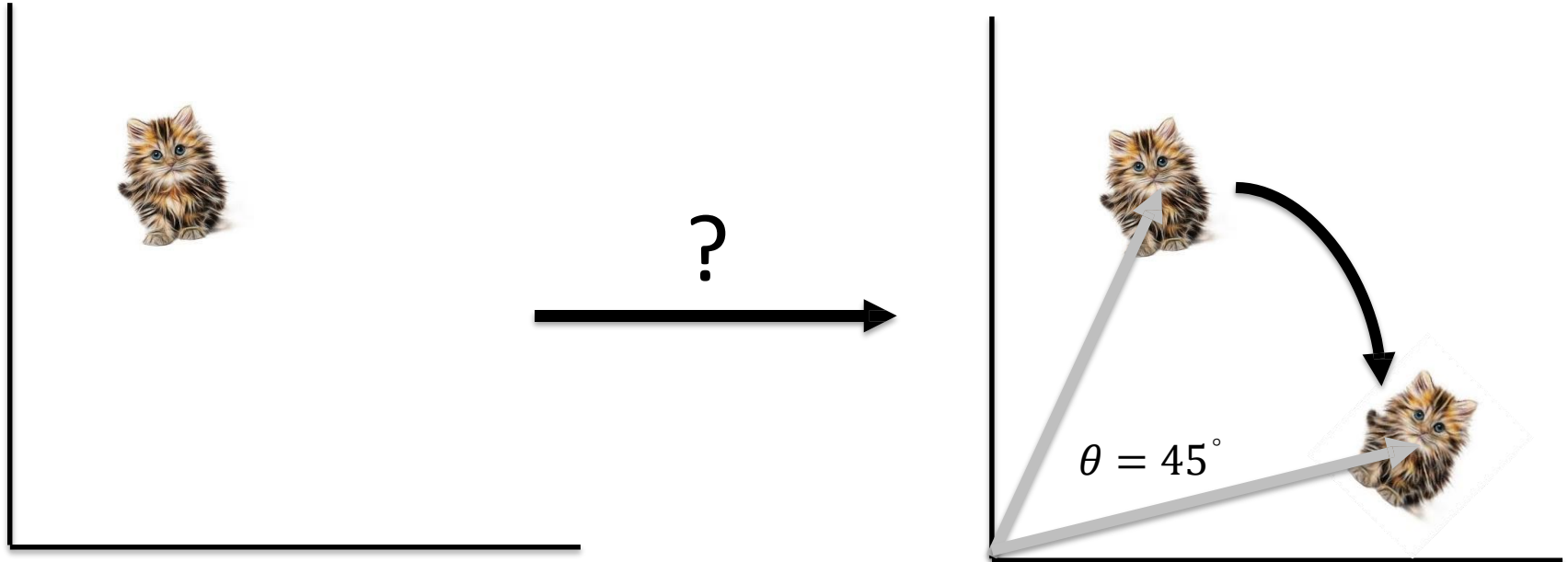
Transformation Matrices

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \times \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} s_x x \\ s_y y \end{bmatrix}$$

$A \qquad p \qquad p'$

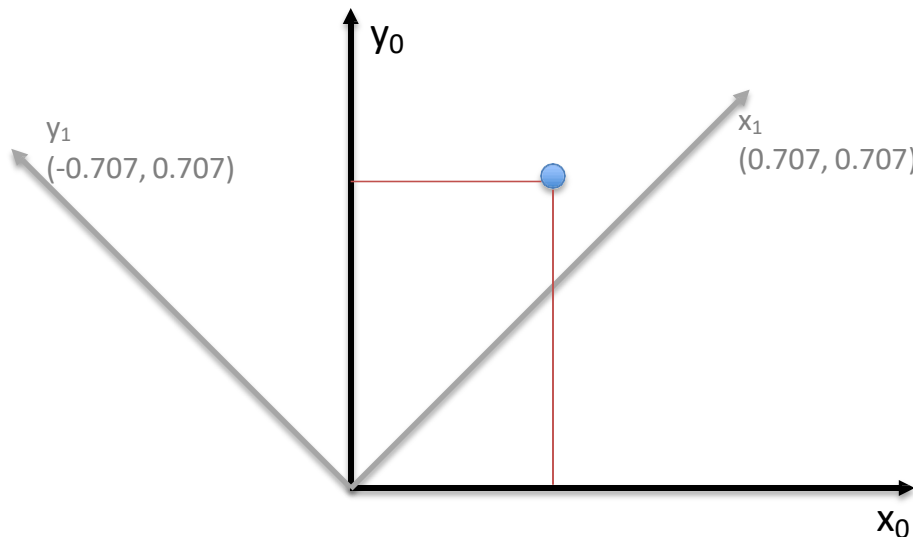


Rotation



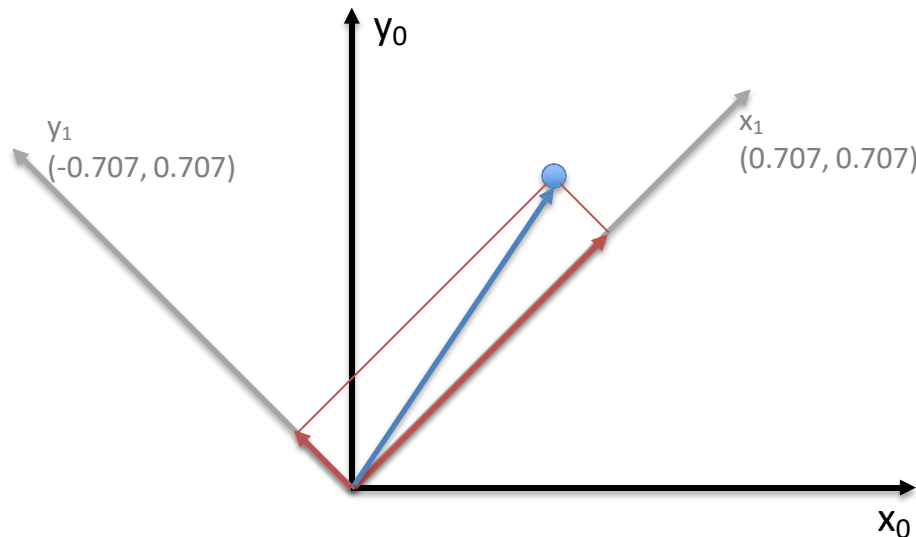
Rotation

- *How can you convert a vector represented in the coordinate frame “0” to a new, rotated coordinate frame “1”?*
- *Remember what a vector is:
[component in direction of the frame’s x axis, component in direction of y axis]*



Rotation

- So to rotate it we must produce this vector:
[component in direction of **new** x axis, component in direction of **new** y axis]
- We can do this easily with dot products!
- New x coordinate is [the new x axis] (x_1) **dot** [original vector]
- New y coordinate is [the new y axis] (y_1) **dot** [original vector]

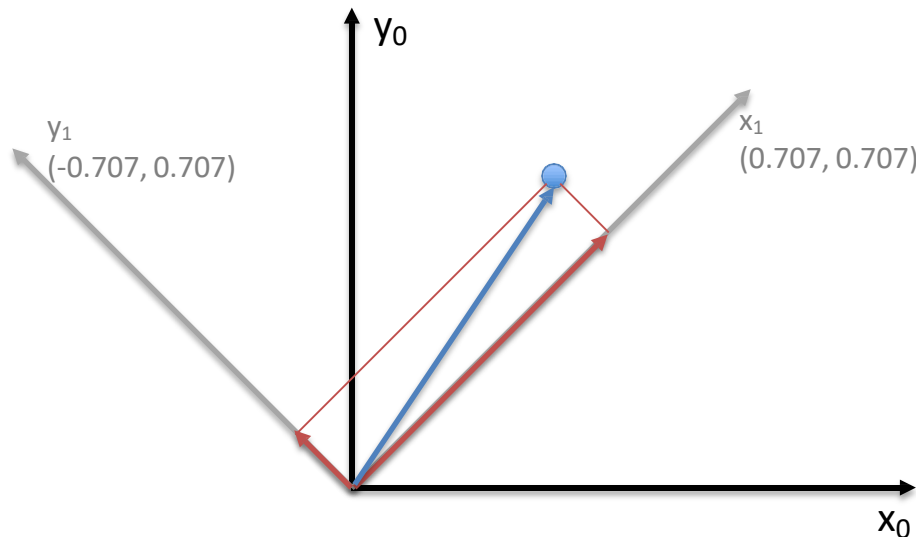


Rotation

- *Insight: something similar happens in a matrix*vector multiplication!*
- *The resulting x coordinate, x' , is: [matrix row 1] dot [original vector]*

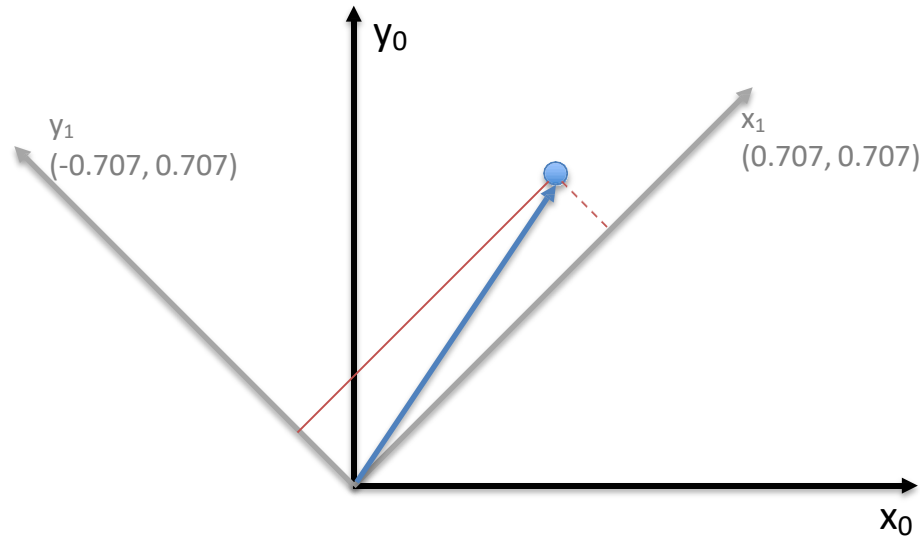
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 0.707 & 0.707 \\ -0.707 & 0.707 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

- *The matrix multiplication $Rp = p'$ produces the coordinates in the new frame.*



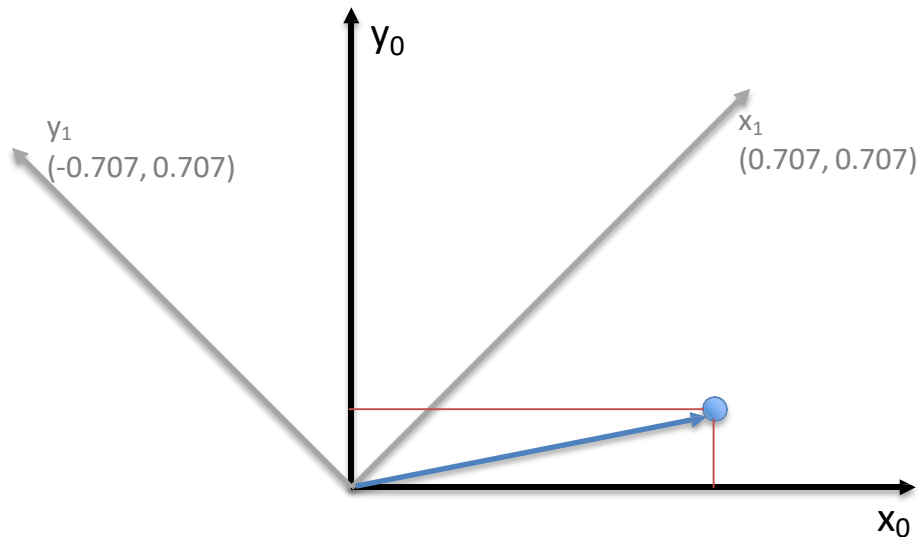
Rotation

- Now we have our point in the new coordinate system which is rotated left
- If we plot the result in the **original** coordinate system, we have rotated the point right



Rotation

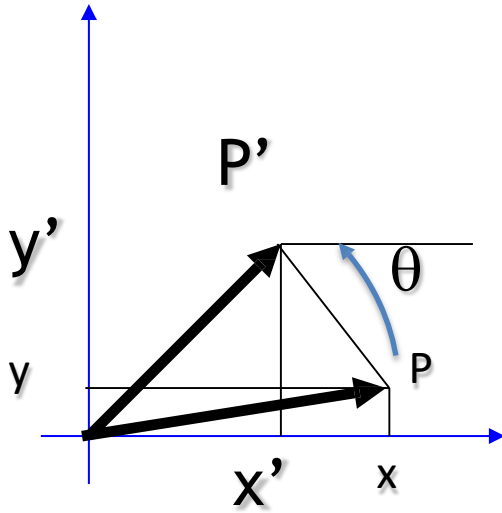
- Now we have our point in the new coordinate system which is rotated left
- If we plot the result in the **original** coordinate system, we have rotated the point right



– Thus, rotation matrices can be used to rotate vectors. We'll usually think of them in that sense-- as operators to rotate vectors

2D Rotation Matrix Formula

Counter-clockwise rotation by an angle θ



$$x' = \cos \theta \, x - \sin \theta \, y$$

$$y' = \cos \theta \, y + \sin \theta \, x$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$p' = R p$$

Transformation Matrices

- *Multiple transformation matrices can be used to transform a point:*

$$p' = R_2 R_1 S p$$

- *The effect of this is to apply their transformations one after the other, from **right to left**.*
- *In the example above, the result is equivalent to*

$$p' = R_2(R_1(Sp))$$

- *The result is exactly the same if we multiply the matrices first, to form a single transformation matrix:*

$$p' = (R_2 R_1 S) p$$

2D transformations

- *Transformation Matrices*
- *Homogeneous coordinates*
- *Translation*
- *Scaling*
- *Rotation*

Homogeneous coordinates

- *In general, a matrix multiplication lets us linearly combine components of a vector*

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

- *This is sufficient for scaling, rotating, and skewing (倾斜) transformations.*
- *But notice, we can't add a constant! ☹*
- *That means, we cannot produce a new (translated) vector $\begin{bmatrix} x + k \\ y + k \end{bmatrix}$.*

Homogeneous coordinates

- *The (somewhat hacky) solution? Stick a “1” at the end of every vector:*

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} ax + by + c \\ dx + ey + f \\ 1 \end{bmatrix}$$

- *Now we can rotate, scale, and skew like before, **AND translate** (note how the multiplication works out, above)*
- *This is called “homogeneous coordinates”*

Homogeneous coordinates

- In homogeneous coordinates, the multiplication works out so the rightmost column of the matrix is a vector that gets added.*

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} ax + by + c \\ dx + ey + f \\ 1 \end{bmatrix}$$

- Generally, a homogeneous transformation matrix will have a bottom row of $[0 \ 0 \ 1]$, so that the result has a “1” at the bottom too.*

Homogeneous coordinates

- *One more thing we might want: to divide the result by something*
 - *For example, we may want to divide by a coordinate, to make things scale down as they get farther away in an image*
 - *Matrix multiplication can't actually divide*
 - *So, **by convention**, in homogeneous coordinates, we'll divide the result by its last coordinate after doing a matrix multiplication*

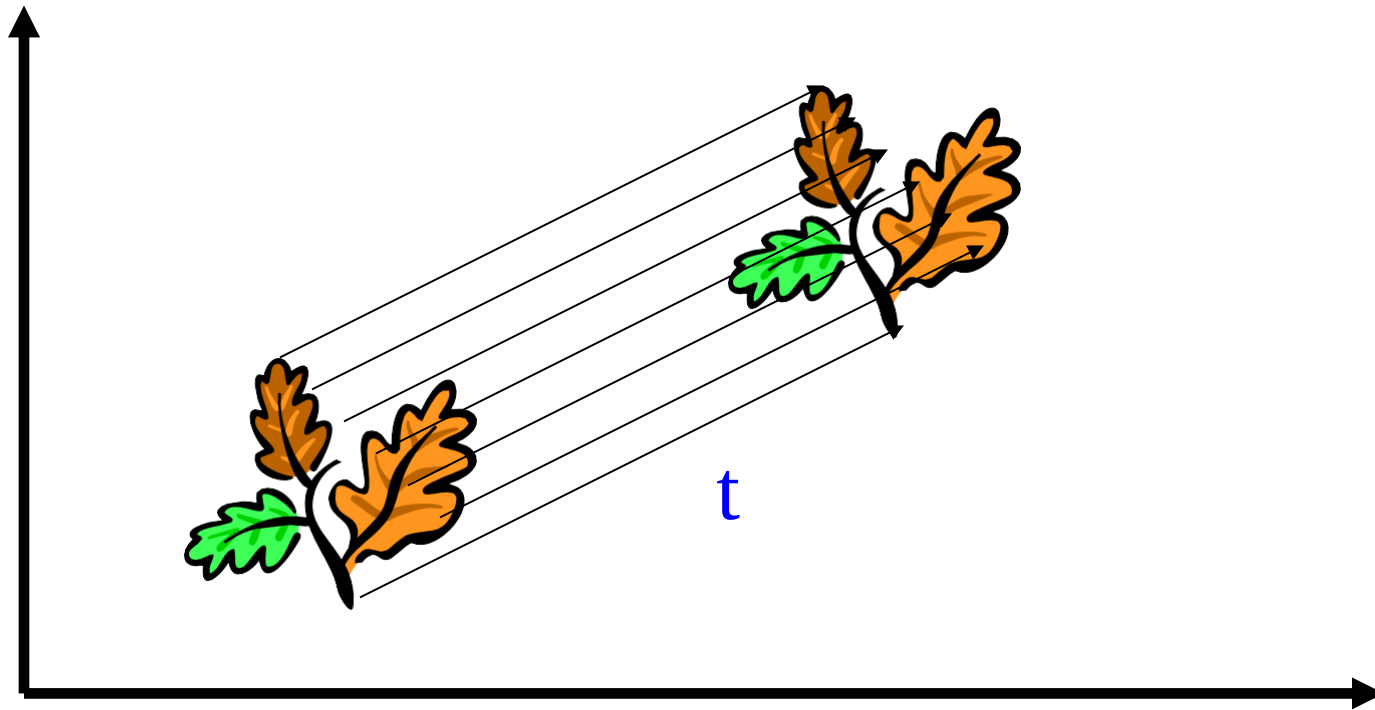
$$\begin{bmatrix} x \\ y \\ 7 \end{bmatrix} \Rightarrow \begin{bmatrix} x/7 \\ y/7 \\ 1 \end{bmatrix}$$

- *The original Cartesian coordinates are recovered by dividing the first two positions by the third.*
- *Unlike Cartesian coordinates, a single point can be represented by infinitely many homogeneous coordinates.*

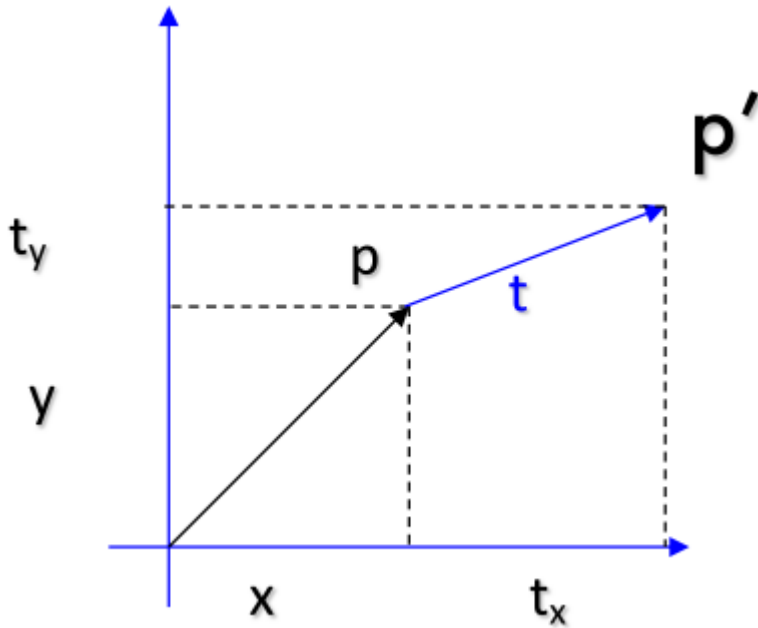
2D transformations

- *Transformation Matrices*
- *Homogeneous coordinates*
- *Translation*
- *Scaling*
- *Rotation*

2D Translation



2D Translation using Homogeneous Coordinates



$$p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

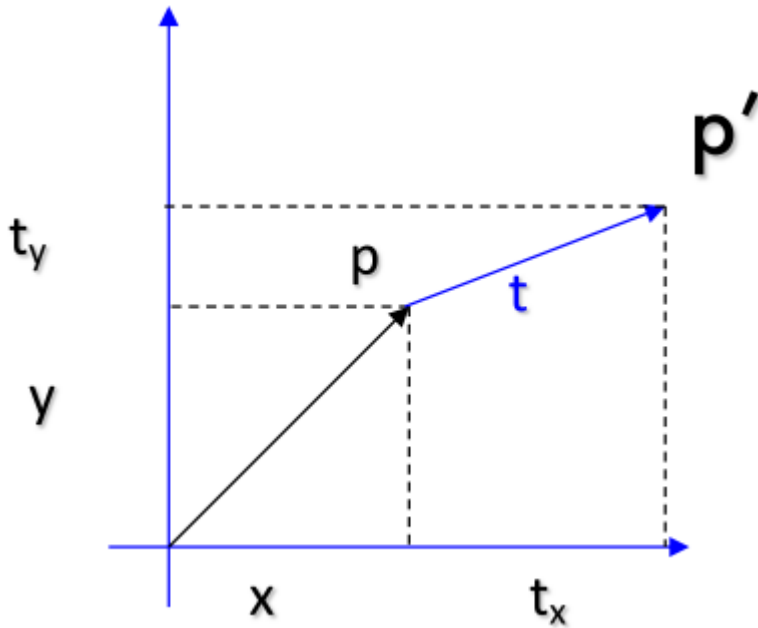
$$t = \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix}$$

$$p' = Tp$$

$$p' \rightarrow \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D Translation using Homogeneous Coordinates



$$p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

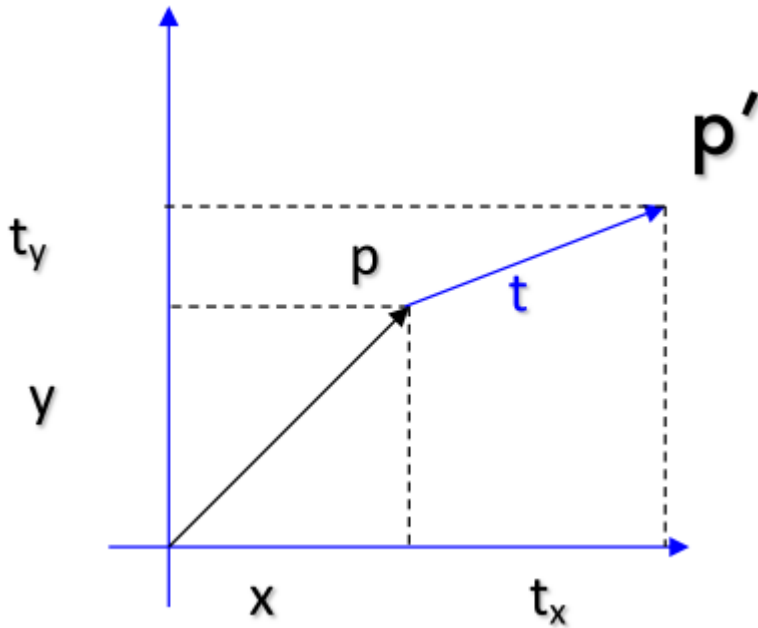
$$t = \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix}$$

$$p' = Tp$$

$$p' \rightarrow \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D Translation using Homogeneous Coordinates



$$p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

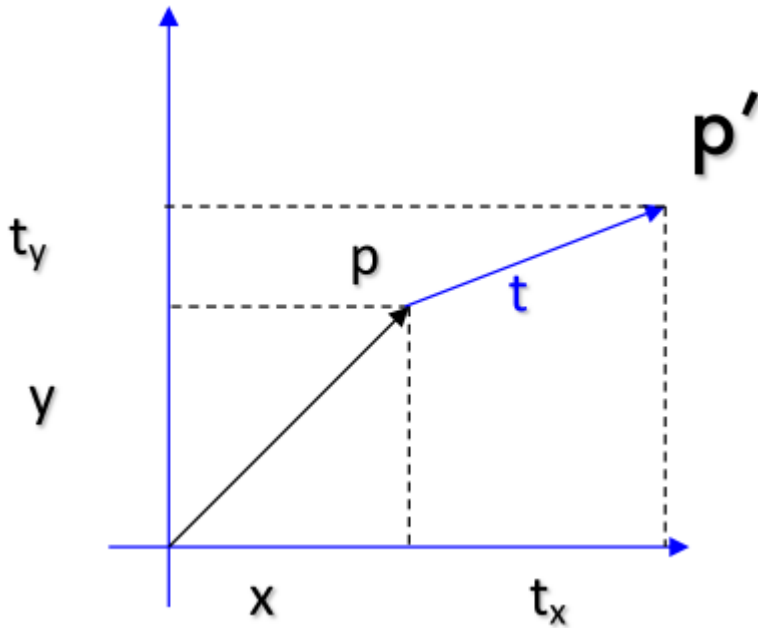
$$t = \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix}$$

$$p' = Tp$$

$$p' \rightarrow \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D Translation using Homogeneous Coordinates



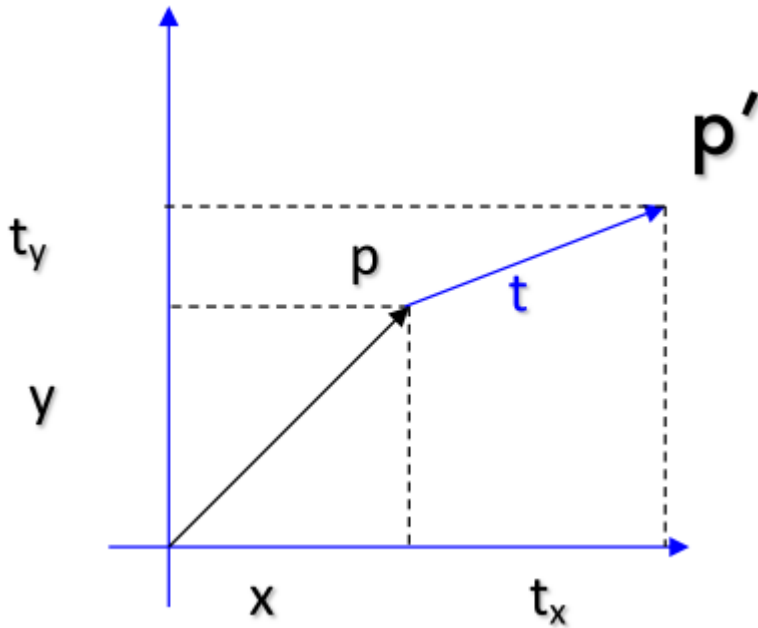
$$p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$t = \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix}$$

$$p' = Tp$$

$$p' \rightarrow \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D Translation using Homogeneous Coordinates



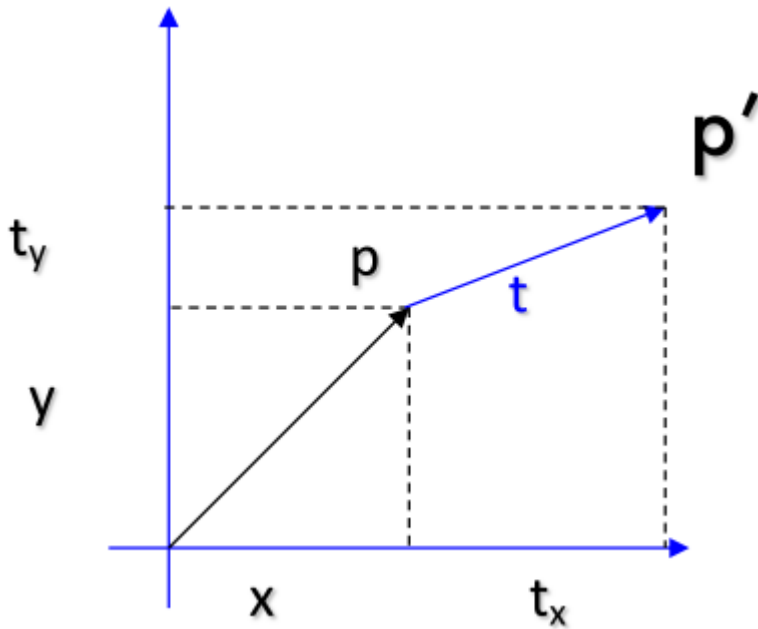
$$p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$t = \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix}$$

$$p' = Tp$$

$$p' \rightarrow \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D Translation using Homogeneous Coordinates



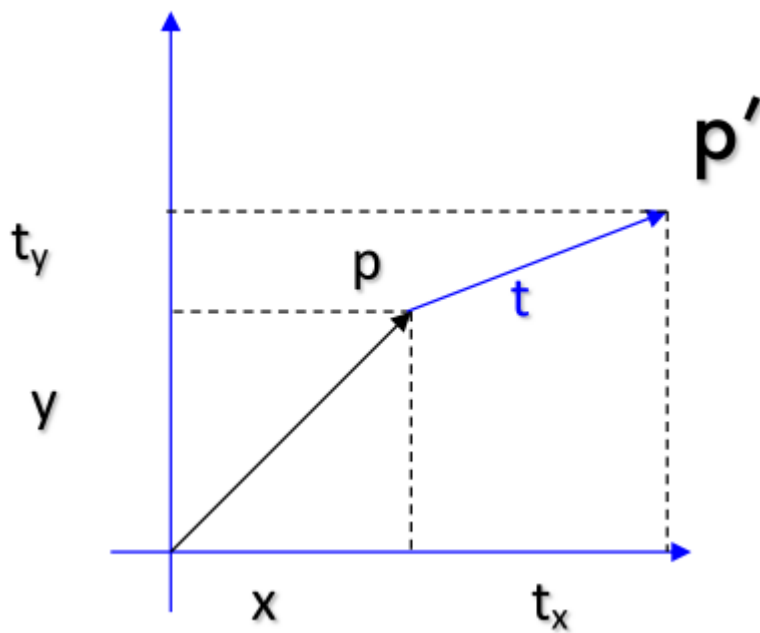
$$p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$t = \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix}$$

$$p' = Tp$$

$$p' \rightarrow \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D Translation using Homogeneous Coordinates



$$p = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$t = \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} t_x \\ t_y \\ 1 \end{bmatrix}$$

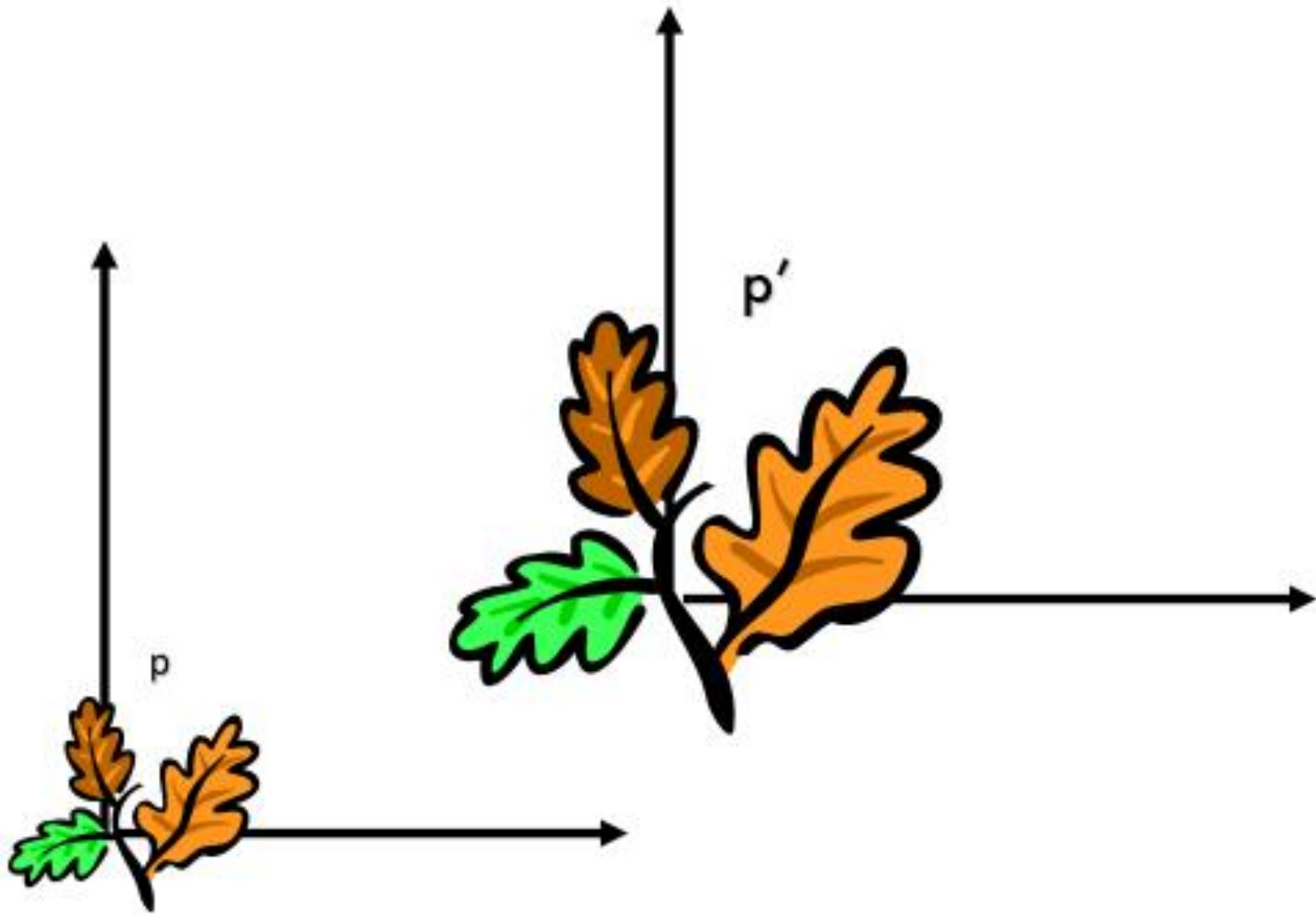
$$p' = Tp$$

$$p' \rightarrow \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} I & t \\ 0 & 1 \end{bmatrix} p = Tp$$

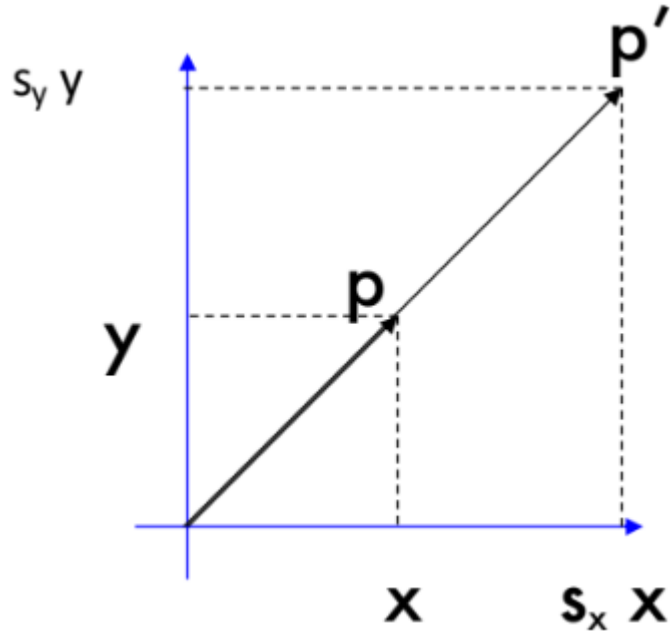
2D transformations

- *Transformation Matrices*
- *Homogeneous coordinates*
- *Translation*
- *Scaling*
- *Rotation*

Scaling



Scaling Equation



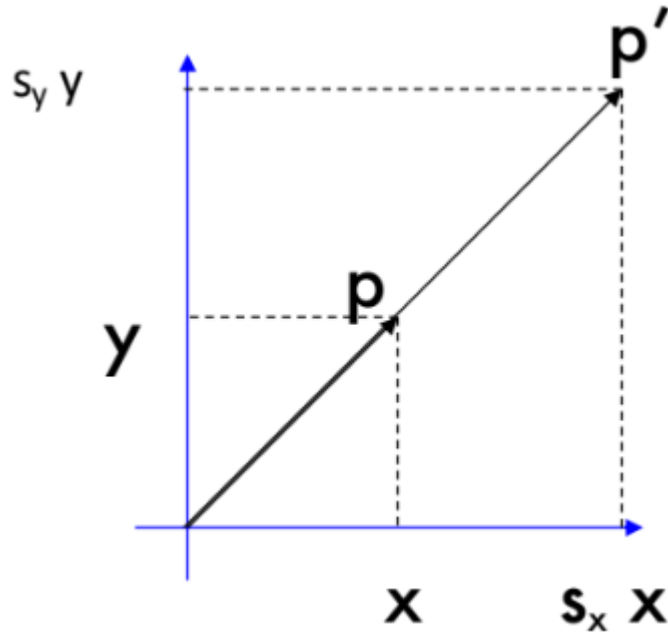
$$p = \begin{bmatrix} x \\ y \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p' = \begin{bmatrix} s_x x \\ s_y y \end{bmatrix} \rightarrow \begin{bmatrix} s_x x \\ s_y y \\ 1 \end{bmatrix}$$

$$p' = Sp$$

$$p' \rightarrow \begin{bmatrix} s_x x \\ s_y y \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scaling Equation



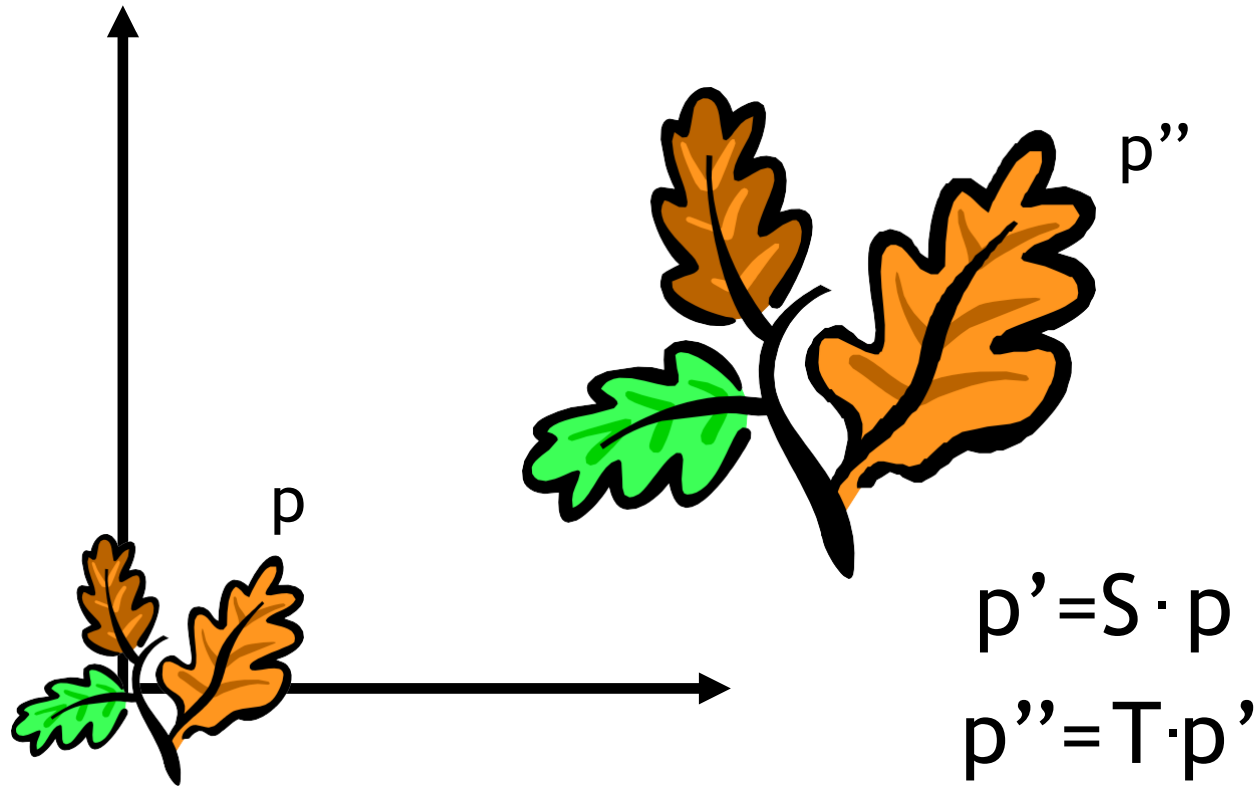
$$p = \begin{bmatrix} x \\ y \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p' = \begin{bmatrix} s_x x \\ s_y y \end{bmatrix} \rightarrow \begin{bmatrix} s_x x \\ s_y y \\ 1 \end{bmatrix}$$

$$p' = Sp$$

$$p' \rightarrow \begin{bmatrix} s_x x \\ s_y y \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} s' & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} p = Sp$$

Scaling & Translating



$$p'' = T \cdot p' = T \cdot (S \cdot p) = T \cdot S \cdot p$$

Scaling & Translating

$$p'' = TSp = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & t_x \\ 0 & s_y & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} S' & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} s_x x + t_x \\ s_y y + t_y \\ 1 \end{bmatrix}$$

Scaling & Translating $\neq$ Translating & Scaling

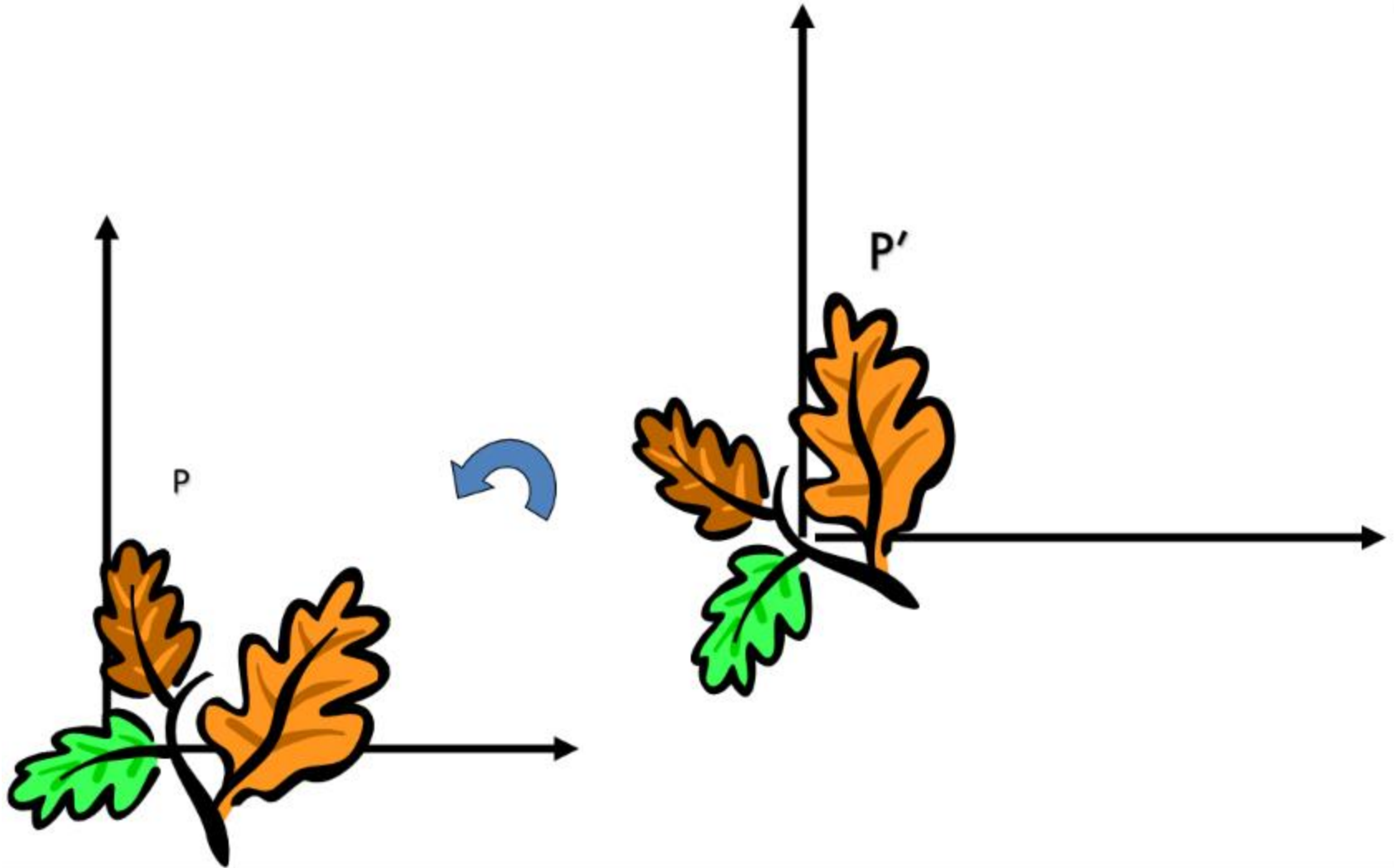
$$p'' = TSp = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & t_x \\ 0 & s_y & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} s_x x + t_x \\ s_y y + t_y \\ 1 \end{bmatrix}$$

$$p''' = STp = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & s_x t_x \\ 0 & s_y & s_y t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} s_x x + s_x t_x \\ s_y y + s_y t_y \\ 1 \end{bmatrix}$$

2D transformations

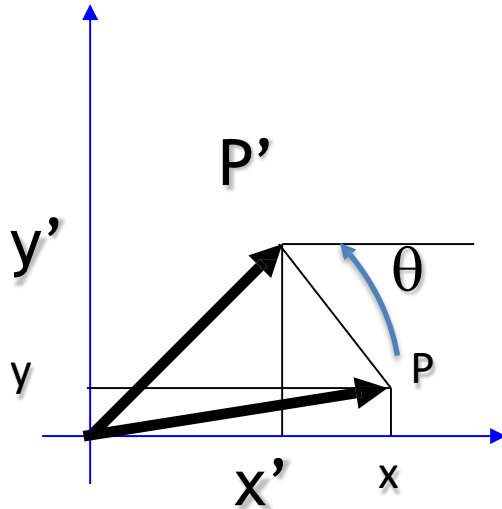
- *Transformation Matrices*
- *Homogeneous coordinates*
- *Translation*
- *Scaling*
- *Rotation*

Rotation



2D Rotation Matrix Formula

Counter-clockwise rotation by an angle θ



$$x' = \cos \theta x - \sin \theta y$$

$$y' = \cos \theta y + \sin \theta x$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$p' = Rp$$

Rotation Matrix Properties

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

A 2D rotation matrix is 2x2

Note: R belongs to the category of normal matrices and satisfies many interesting properties:

$$\mathbf{R} \cdot \mathbf{R}^T = \mathbf{R}^T \cdot \mathbf{R} = \mathbf{I}$$

$$\det(\mathbf{R}) = 1$$

Rotation Matrix Properties

- *Transpose of a rotation matrix produces a rotation in the opposite direction*

$$\mathbf{R} \cdot \mathbf{R}^T = \mathbf{R}^T \cdot \mathbf{R} = \mathbf{I}$$

$$\det(\mathbf{R}) = 1$$

- *The rows of a rotation matrix are always mutually perpendicular (a.k.a. orthogonal) unit vectors*
– *(and so are its columns)*

Scaling + Rotation + Translation

$$p' = (T R S) p$$

$$p' = TRSp = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} S & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \boxed{\begin{bmatrix} RS & t \\ 0 & 1 \end{bmatrix}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

This is the form of the
general-purpose
transformation matrix

Summary

- *2D transformations*
 - *Transformation Matrices*
 - *Homogeneous coordinates*
 - *Translation*
 - *Scaling*
 - *Rotation*

Color and Filters

What we will learn today?

- Color
- Image histograms
- Images as functions
- Linear systems (filters)
- Convolution and correlation

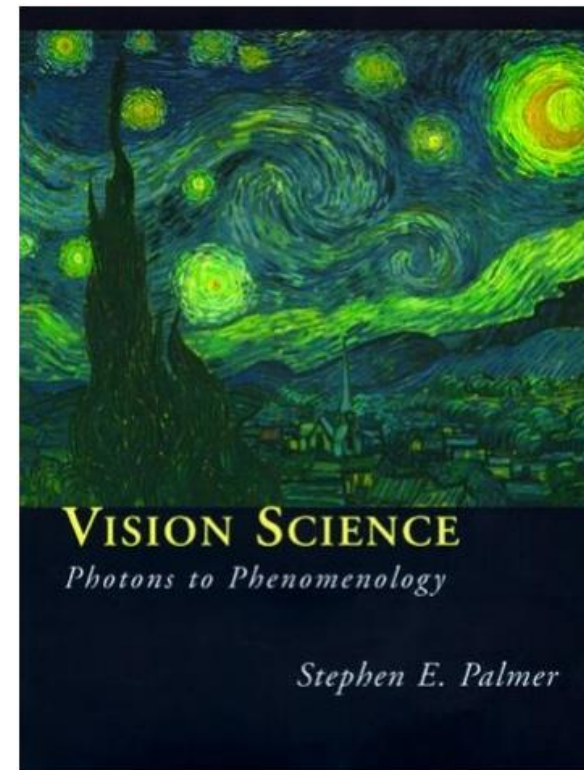


What we will learn today?

- Color
- Image histograms
- Images as functions
- Linear systems (filters)
- Convolution and correlation

What is color?

- *The result of interaction between physical light in the environment and our visual system.*
- A *psychological property* of our visual experiences when we look at objects and lights, *not a physical property* of those objects or lights.



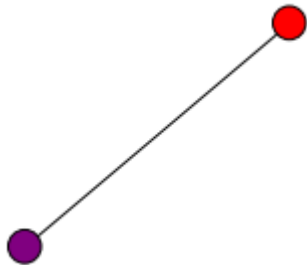
Interaction of light and surfaces

- *What is the observed color of any surface under monochromatic light?*

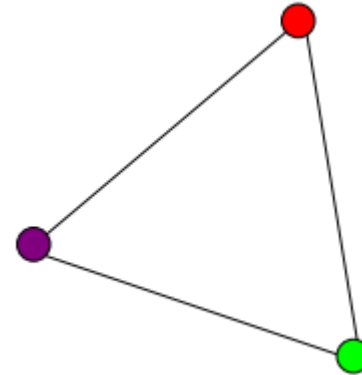


Linear color spaces

- *Defined by a choice of three primaries (三原色) .*
- *The coordinates of a color are given by the weights of the primaries used to match it.*



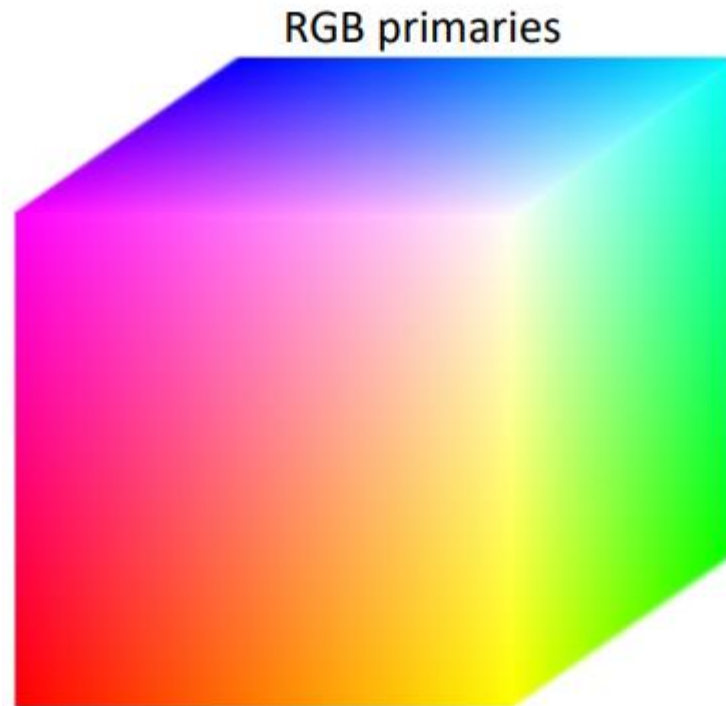
mixing two lights produces colors that lie along a straight line in color space



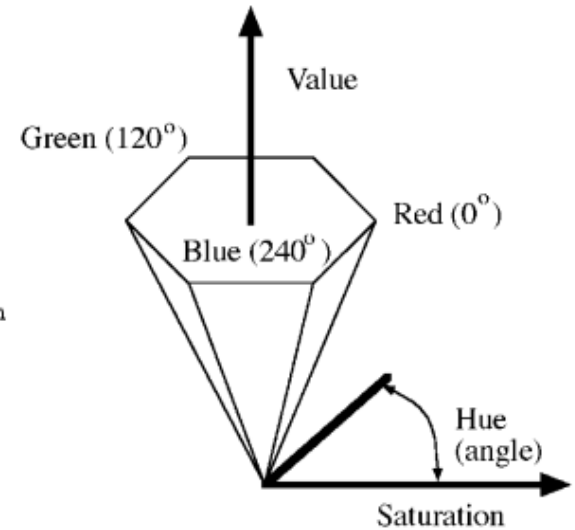
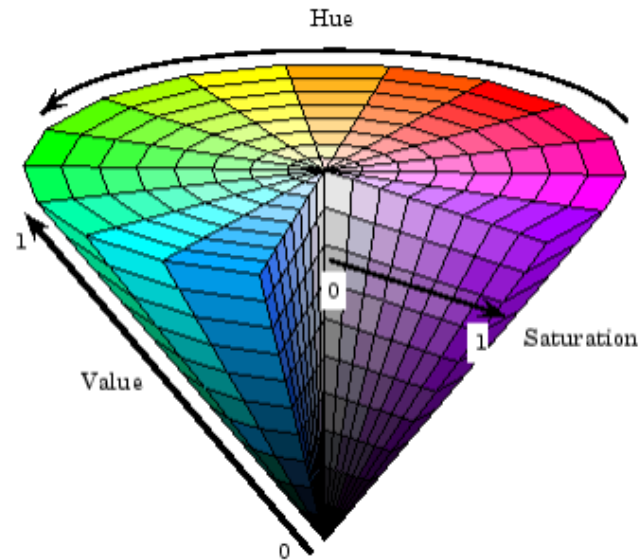
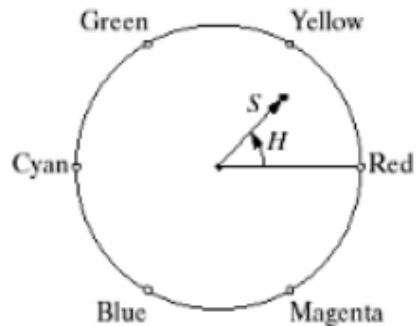
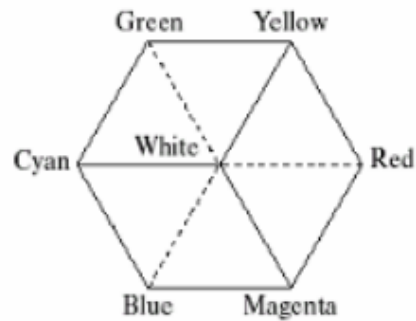
mixing three lights produces colors that lie within the triangle they define in color space

RGB space

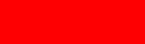
- *Primaries are monochromatic lights (for monitors, they correspond to the three types of phosphors)*



Nonlinear color spaces: HSV

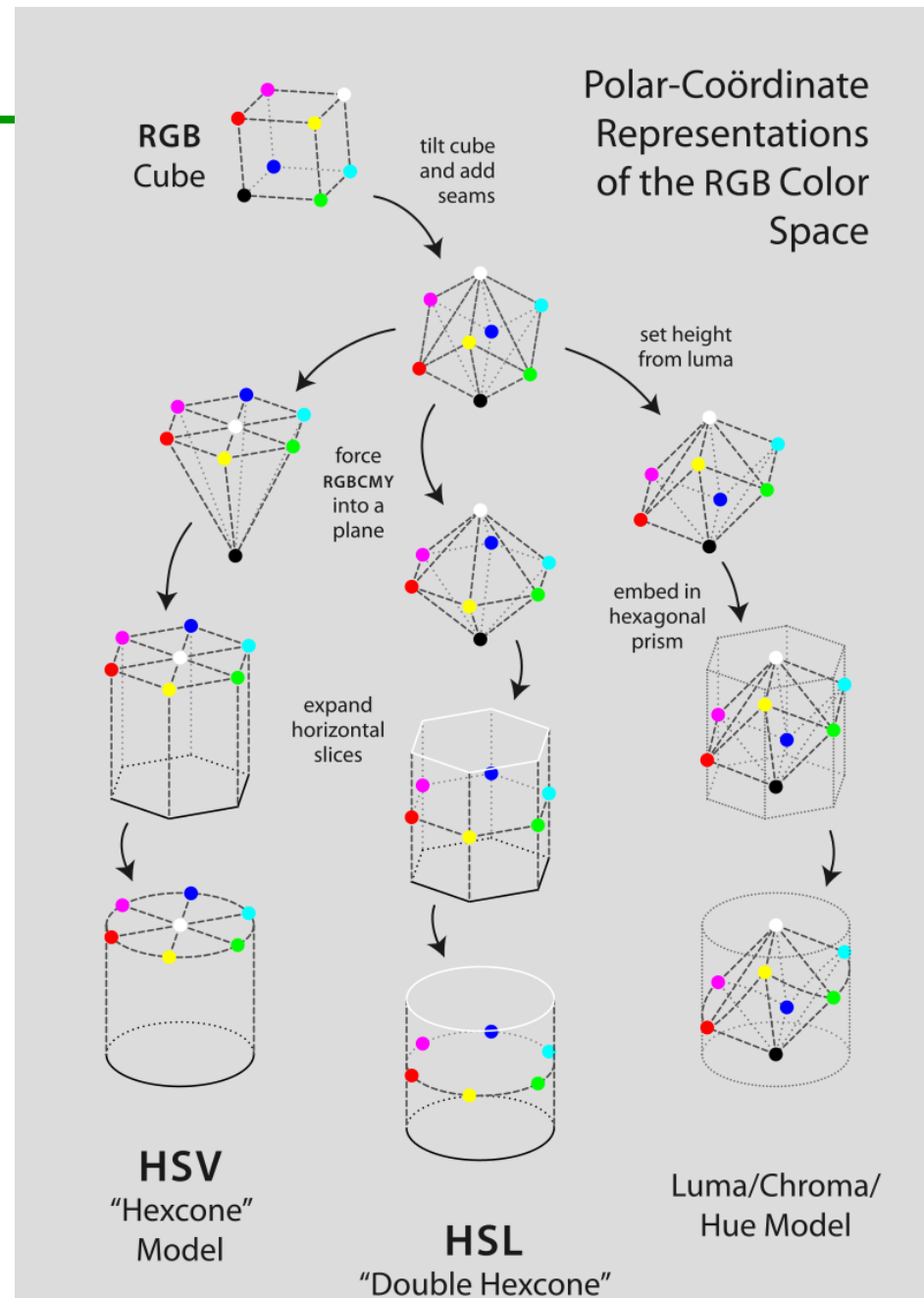


Examples:

RGB	HSL	HSV	结果
(1, 0, 0)	(0°, 1, 0.5)	(0°, 1, 1)	
(0.5, 1, 0.5)	(120°, 1, 0.75)	(120°, 0.5, 1)	
(0, 0, 0.5)	(240°, 1, 0.25)	(240°, 1, 0.5)	

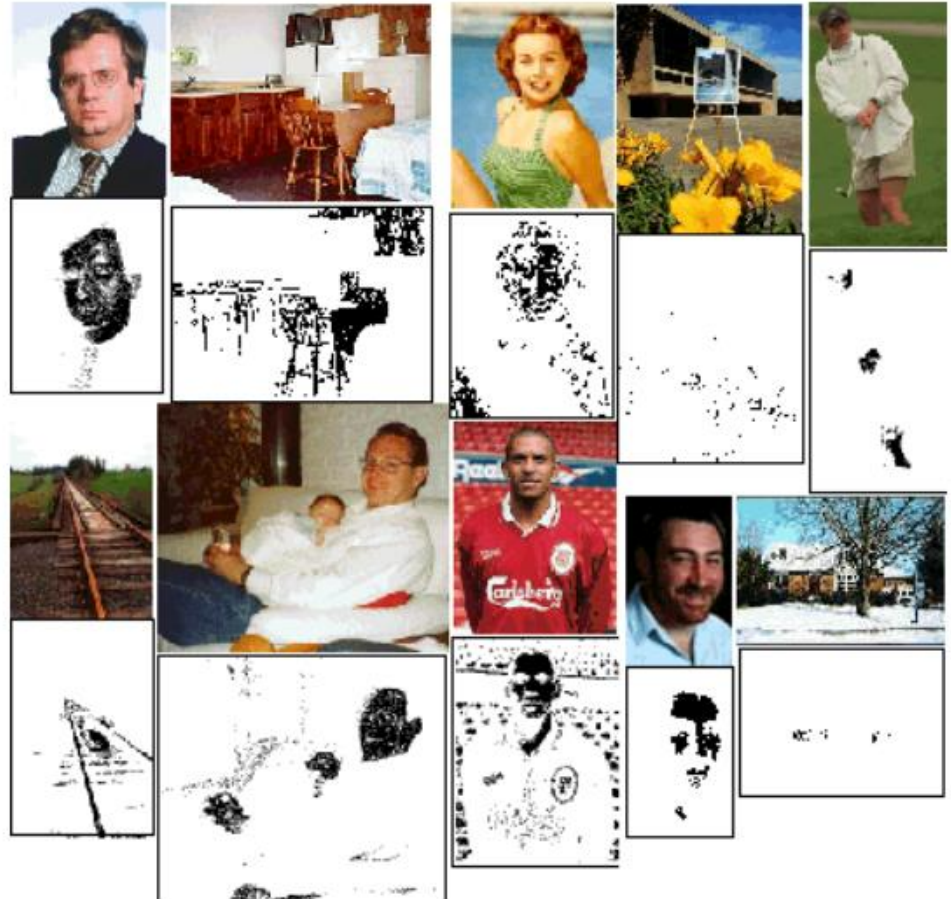
- *Perceptually meaningful dimensions: Hue (色调), Saturation (饱和度), Value (明度)*
- *RGB cube on its vertex*

Other color spaces



Uses of color in computer vision

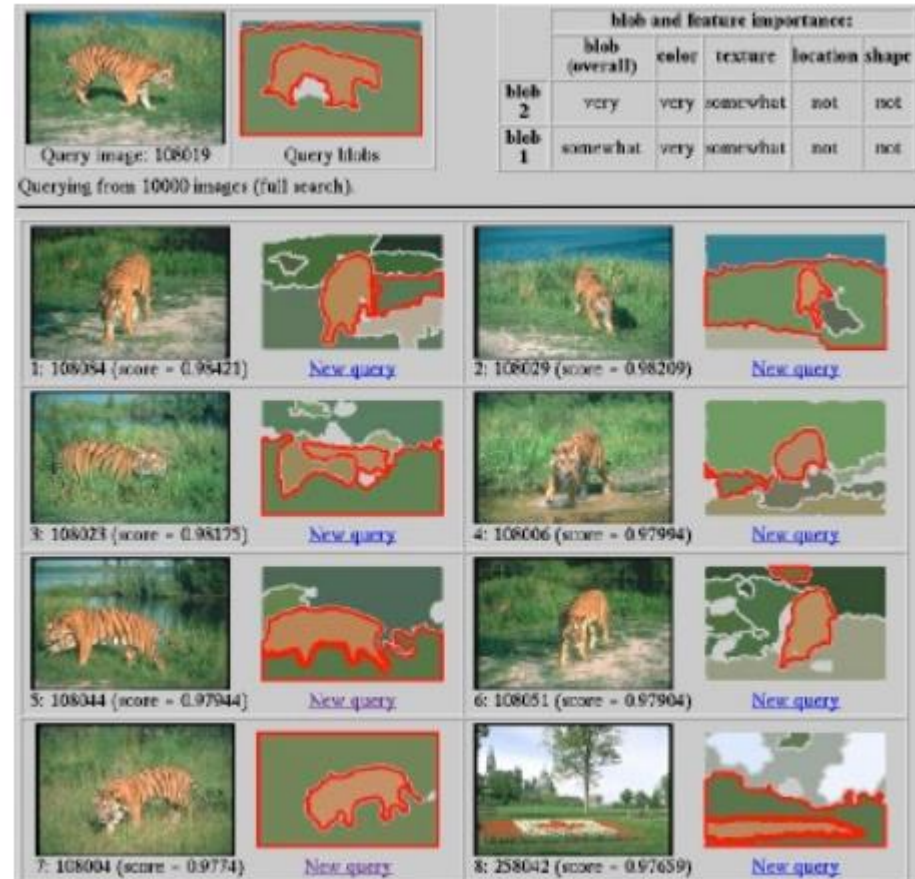
Skin detection



M. Jones and J. Rehg, Statistical Color Models with Application to Skin Detection, IJCV 2002.

Uses of color in computer vision

Image segmentation and retrieval



C. Carson, S. Belongie, H. Greenspan, and Ji. Malik, Blobworld: Image segmentation using Expectation-Maximization and its application to image querying, ICVIS 1999.



What we will learn today?

- Color
- Image histograms
- Images as functions
- Linear systems (filters)
- Convolution and correlation

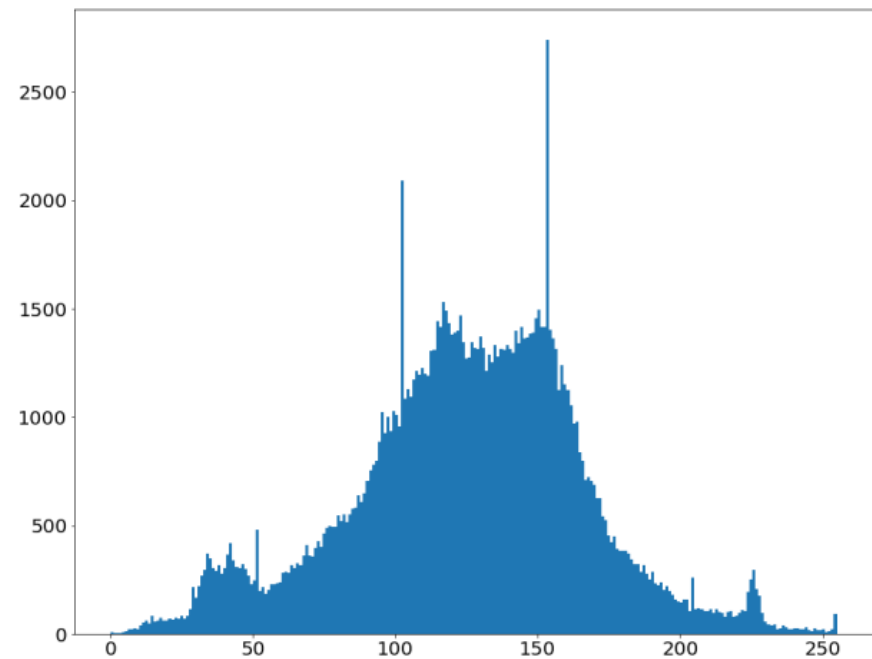
Histograms

- *Histogram of an image provides the frequency of the brightness (intensity) value in the image.*

```
def histogram(im):  
    h = np.zeros(255)  
    for row in im.shape[0]:  
        for col in im.shape[1]:  
            val = im[row, col]  
            h[val] += 1
```

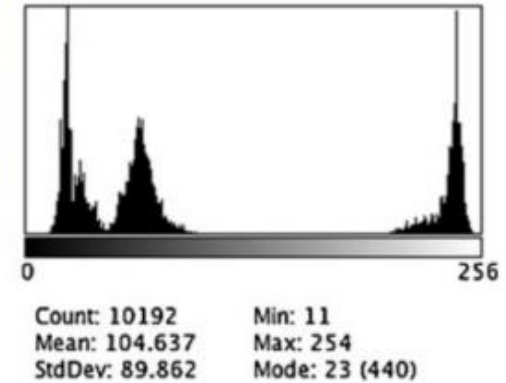
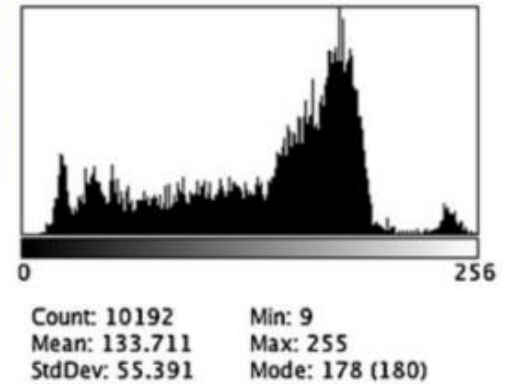
Histograms

- *Histogram captures the distribution of gray levels in the image.*



Histograms

- *Histogram captures the distribution of gray levels in the image.*





What we will learn today?

- Color
- Image histograms
- **Images as functions**
- Linear systems (filters)
- Convolution and correlation

Images as discrete functions

- Images are usually **digital (discrete)**:
 - Sample** the 2D space on a regular grid
- Represented as a matrix of integer values

pixel



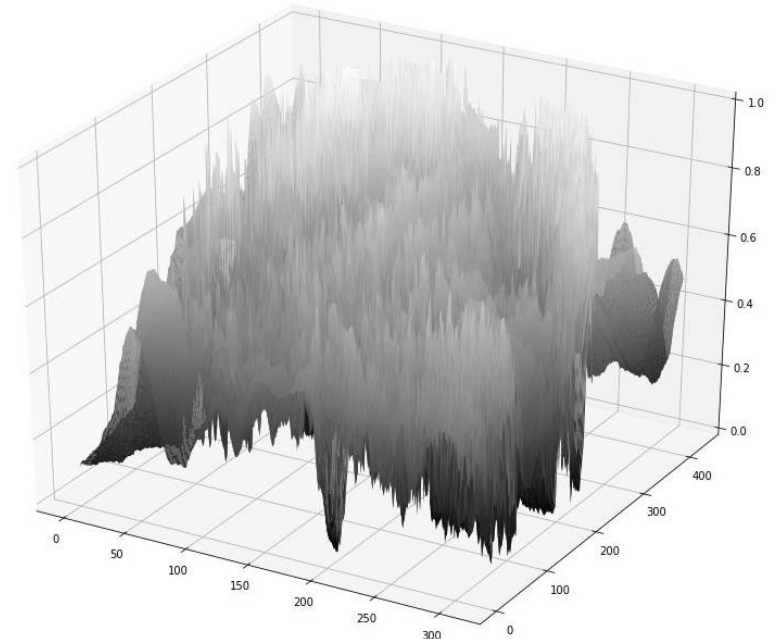
	m	62	79	23	119	120	05	4	0
n		10	10	9	62	12	78	34	0
		10	58	197	46	46	0	0	48
		176	135	5	188	191	68	0	49
		2	1	1	29	26	37	0	77
		0	89	144	147	187	102	62	208
		255	252	0	166	123	62	0	31
		166	63	127	17	1	0	99	30

Images as discrete functions

- **An Image** as a function f from $\mathbb{R}^2$ to $\mathbb{R}^M$:
 - $f[x, y]$ gives the **intensity** at position $[x, y]$
 - Defined over a rectangle, with a finite range:
 $f: [a, b] \times [c, d] \rightarrow [0, 255]$

Domain
support

range



Images as discrete functions

- **An Image** as a function f from $\mathbb{R}^2$ to $\mathbb{R}^M$:
 - $f[x, y]$ gives the **intensity** at position $[x, y]$
 - Defined over a rectangle, with a finite range:
 $f: [a, b] \times [c, d] \rightarrow [0, 255]$

Domain
support

range

- A color image:

$$f(x, y) = \begin{bmatrix} r(x, y) \\ g(x, y) \\ b(x, y) \end{bmatrix}$$



Phil Noble / AP



What we will learn today?

- Color
- Image histograms
- Images as functions
- **Linear systems (filters)**
- Convolution and correlation

Applications of Linear Filters (Systems)

De-nosing



Super-resolution



Inpainting



Systems or Filters

- **Filtering:** *Forming a new image whose pixel values are transformed from original pixel values*
- **Goals:**
 1. *Extract useful information from images: Features (edges, corners, blobs...)*
 2. *Transform images to modify/enhance image properties: Super-resolution; in-painting; de-noising*

Systems or Filters

- We define a system as a unit that converts an input function $f[n,m]$ into an output function $g[n,m]$, where $[n, m]$ are the independent variables.
 - In the case of images, $[n, m]$ represents the **spatial position in the image**.

$$f[n, m] \longrightarrow \boxed{\text{System } \mathcal{S}} \longrightarrow g[n, m]$$

Systems and Filters

- S is the **system operator**, defined as a mapping/assignment that transforms the input f into the output g .

$$f[n, m] \rightarrow \boxed{\text{System } \mathcal{S}} \rightarrow g[n, m]$$

$$g = \mathcal{S}[f], \quad g[n, m] = \mathcal{S}\{f[n, m]\}$$

$$f[n, m] \xrightarrow{\mathcal{S}} g[n, m]$$

Filter example #1: Moving Average

- 2D moving average over a 3×3 neighborhood window

Original image



Smoothed image



Filter example #1: Moving Average

- 2D moving average over a 3×3 window of neighborhood

Sum over rows

Sum over columns

$$\begin{aligned}
 g[n, m] &= \frac{1}{9} \sum_{k=n-1}^{n+1} \sum_{l=m-1}^{m+1} f[k, l] \\
 &= \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 f[n-k, m-l]
 \end{aligned}$$

h

	1	1	1
$\frac{1}{9}$	1	1	1
	1	1	1

Filter example #1: Moving Average

$f[n, m]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[n, m]$

Courtesy of S. Seitz

Filter example #1: Moving Average

$f[n, m]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[n, m]$

	0	10							

Courtesy of S. Seitz

Filter example #1: Moving Average

$f[n, m]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[n, m]$

	0	10	20						

Courtesy of S. Seitz

Filter example #1: Moving Average

$f[n, m]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[n, m]$

	0	10	20	30					

Courtesy of S. Seitz

Filter example #1: Moving Average

$f[n, m]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[n, m]$

	0	10	20	30	30				

Courtesy of S. Seitz

Filter example #1: Moving Average

$f[n, m]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[n, m]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

Courtesy of S. Seitz

Filter example #1: Moving Average

- **In summary:**

1. *This filter “transforms” each pixel value into the average value of its neighborhood.*
2. *Achieve smoothing effect (remove sharp features)*

$$\frac{1}{9} h[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

Filter example #1: Moving Average

- 2D moving average over a 3×3 neighborhood window

Original image



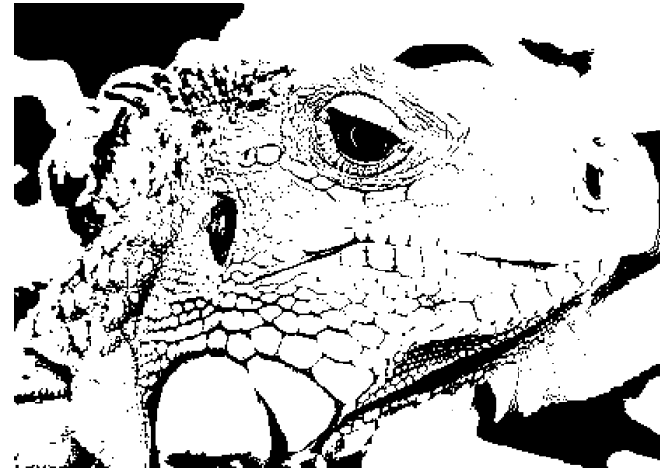
Smoothed image



Filter example #2: Image Segmentation

- Image segmentation based on a simple threshold:

$$g[n, m] = \begin{cases} 255, & f[n, m] > 100 \\ 0, & \text{otherwise.} \end{cases}$$



Properties of systems

- Amplitude properties:

- Additivity

$$S[f_i[n, m] + f_j[n, m]] = S[f_i[n, m]] + S[f_j[n, m]]$$

- Homogeneity

$$S[\alpha f_i[n, m]] = \alpha S[f_i[n, m]]$$

- Superposition

$$S[\alpha f_i[n, m] + \beta f_j[n, m]] = \alpha S[f_i[n, m]] + \beta S[f_j[n, m]]$$

- Stability

$$|f[n, m]| \leq k \implies |g[n, m]| \leq ck$$

- Invertibility

$$S^{-1}[S[f_i[n, m]]] = f[n, m]$$

Properties of systems

- Spatial properties

- Causality

$$\text{for } n < n_0, m < m_0, \text{ if } f[n, m] = 0 \implies g[n, m] = 0$$

- Shift invariance

$$f[n - n_0, m - m_0] \xrightarrow{\mathcal{S}} g[n - n_0, m - m_0]$$

Shift-Invariant (SI) Systems

- We have $f[n, m] \xrightarrow{\mathcal{S}} g[n, m]$

– if for every input image $f[n, m]$ and shifts n_0, m_0

$$f[n - n_0, m - m_0] \xrightarrow{\mathcal{S}} g[n - n_0, m - m_0]$$

– then, $\mathcal{S}$ is Shift Invariant

Shift-Invariant (SI) Systems

- Is the moving average system shift invariant?

$$g[n, m] = \frac{1}{9} \sum_{k=n-1}^{n+1} \sum_{l=m-1}^{m+1} f[k, l]$$

$f[n, m]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[n, m]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

Shift-Invariant (SI) Systems

- Is the moving average system shift invariant?
 - Let's start with passing f through our system

$$f[n, m] \xrightarrow{S} g[n, m]$$

$$g[n, m] = \frac{1}{9} \sum_{k=n-1}^{n+1} \sum_{l=m-1}^{m+1} f[k, l] = \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 f[n - k, m - l]$$

- Now, let's see what we get when we pass in a shifted version of the input f :

$$\begin{aligned} f[n - n_0, m - m_0] &\xrightarrow{S} \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 f[(n - n_0) - k, (m - m_0) - l] \\ &= g[n - n_0, m - m_0] \end{aligned}$$

Yes!

Linear Systems (filters)

$$f[n, m] \rightarrow \boxed{\text{System } \mathcal{S}} \rightarrow g[n, m]$$

- Linear filtering:
 - Form a new image whose pixels are a weighted sum of original pixel values
 - Use the same set of weights at each point
- S is a linear system (filter) if and only if it satisfies
$$S\{\alpha f_1[n, m] + \beta f_2[n, m]\} = \alpha S\{f_1[n, m]\} + \beta S\{f_2[n, m]\}$$

superposition property

Linear Systems (filters)

$$S\{\alpha f_1[n, m] + \beta f_2[n, m]\} = \alpha S\{f_1[n, m]\} + \beta S\{f_2[n, m]\}$$

- Is the moving average a linear system? **Yes!**
- Is thresholding a linear system? **No!**
 - We could have $f_1[n, m] + f_2[n, m] > T$, when $f_1[n, m] < T$ and $f_2[n, m] < T$

Linear Shift Invariant (LSI) systems

- An LSI system satisfies two properties:
 - Superposition property

$$S\{\alpha f_1[n, m] + \beta f_2[n, m]\} = \alpha S\{f_1[n, m]\} + \beta S\{f_2[n, m]\}$$

- Shift invariance

$$f[n - n_0, m - m_0] \xrightarrow{S} g[n - n_0, m - m_0]$$

Linear Shift Invariant (LSI) systems

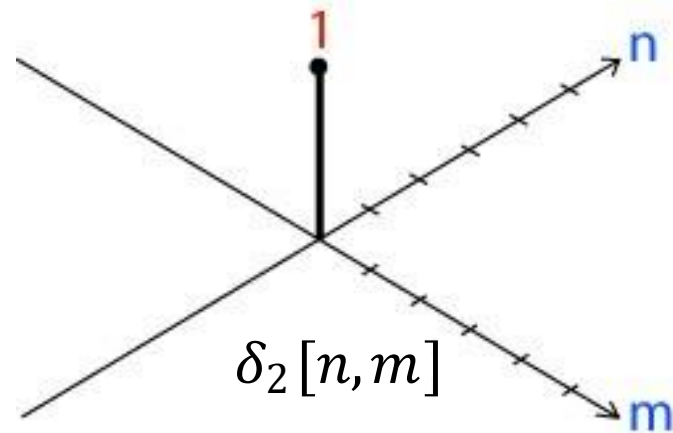
- Moving average system is **shift invariant and linear**
 - Why are linear shift invariant systems important?
- Our visual system is a
linear shift invariant system**
- We are going to use this as an example to dive into interesting properties about linear shift-invariant systems.

Linear Shift Invariant (LSI) systems

- 2D impulse function $\delta_2[n, m]$

1. 1 at $[0,0]$.
2. 0 everywhere else

	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	
	0	0	0	1	0	0	0	
	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	



Linear Shift Invariant (LSI) systems

- Impulse response to the moving average filter
 - Recall the expression for our 3x3 moving average filter:

$$f[n, m] \xrightarrow{S} \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 f[n-k, m-l]$$

- We can use it to obtain an expression for the impulse response

$$\begin{aligned} \delta_2[n, m] &\xrightarrow{S} h[n, m] \\ &= \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 \delta_2[n-k, m-l] \end{aligned}$$

Linear Shift Invariant (LSI) systems

- Impulse response to the moving average filter

		?		
		$h[0,0]$		

$$\begin{aligned} \delta_2 &\xrightarrow{S} h[n,m] \\ &= \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 \delta_2[n-k, m-l] \end{aligned}$$

Linear Shift Invariant (LSI) systems

- Impulse response to the moving average filter

		$\frac{1}{9}$ $h[0,0]$	$?$ $h[0,1]$	

$$\begin{aligned} \delta_2 &\xrightarrow{s} h[n, m] \\ &= \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 \delta_2[n - k, m - l] \end{aligned}$$

Linear Shift Invariant (LSI) systems

- Impulse response to the moving average filter

		$\frac{1}{9}$ $h[0,0]$	$\frac{1}{9}$ $h[0,1]$	
			? $h[1,1]$	

$$\begin{aligned} \delta_2 &\xrightarrow{s} h[n, m] \\ &= \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 \delta_2[n - k, m - l] \end{aligned}$$

Linear Shift Invariant (LSI) systems

- Impulse response to the moving average filter

		$\frac{1}{9}$ $h[0,0]$	$\frac{1}{9}$ $h[0,1]$	
			$\frac{1}{9}$ $h[1,1]$	

$$\begin{aligned} \delta_2 &\xrightarrow{S} h[n, m] \\ &= \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 \delta_2[n - k, m - l] \end{aligned}$$

Linear Shift Invariant (LSI) systems

- Impulse response to the moving average filter

		$\frac{1}{9}$ $h[0,0]$	$\frac{1}{9}$ $h[0,1]$	? $h[0,2]$
			$\frac{1}{9}$ $h[1,1]$	

$$\begin{aligned} \delta_2 &\xrightarrow{S} h[n, m] \\ &= \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 \delta_2[n - k, m - l] \end{aligned}$$

Linear Shift Invariant (LSI) systems

- Impulse response to the moving average filter

		$\frac{1}{9}$ $h[0,0]$	$\frac{1}{9}$ $h[0,1]$	0 $h[0,2]$
			$\frac{1}{9}$ $h[1,1]$	

$$\begin{aligned} \delta_2 &\xrightarrow{S} h[n, m] \\ &= \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 \delta_2[n - k, m - l] \end{aligned}$$

Linear Shift Invariant (LSI) systems

- Impulse response to the moving average filter

0	0	0	0	0
0	$\frac{1}{9}$ $h[-1,-1]$	$\frac{1}{9}$	$\frac{1}{9}$	0
0	$\frac{1}{9}$	$\frac{1}{9}$ $h[0,0]$	$\frac{1}{9}$ $h[0,1]$	0 $h[0,2]$
0	$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$ $h[1,1]$	0
0	0	0	0	0

$$\delta_2 \xrightarrow{s} g[n, m]$$

$$= \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 \delta_2[n - k, m - l]$$

Linear Shift Invariant (LSI) systems

- Impulse response of the 3 by 3 moving average filter

$$h[n, m] = \frac{1}{9} \sum_{k=-1}^1 \sum_{l=-1}^1 \delta_2[n - k, m - l]$$

$$= \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$$

h

	1	1	1
$\frac{1}{9}$	1	1	1
	1	1	1

Linear Shift Invariant (LSI) systems

- An LSI system is completely specified by its impulse response.

$$\boxed{\delta_2[n, m]} \xrightarrow{S} \boxed{h[n, m]}$$

Pass in an impulse function Record its response

- By passing an impulse function into an LSI system, we get its impulse response.

Linear Shift Invariant (LSI) systems

- Key idea: write down f as a sum of impulses
 - Let's say our input f is a 3x3 image:

$$\begin{array}{|c|c|c|} \hline f[0,0] & f[0,1] & f[1,1] \\ \hline f[1,0] & f[1,1] & f[1,2] \\ \hline f[2,0] & f[2,1] & f[2,2] \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline f[0,0] & 0 & 0 \\ \hline 0 & 0 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline 0 & f[0,1] & 0 \\ \hline 0 & 0 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} + \dots + \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 0 & 0 \\ \hline 0 & 0 & f[2,2] \\ \hline \end{array}$$

$$= \begin{array}{|c|c|c|} \hline 1 & 0 & 0 \\ \hline 0 & 0 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \begin{array}{c} f[0,0] \\ * \end{array} + \begin{array}{|c|c|c|} \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \begin{array}{c} f[0,1] \\ * \end{array} + \dots + \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline 0 & 0 & 0 \\ \hline 0 & 0 & 1 \\ \hline \end{array} \begin{array}{c} f[2,2] \\ * \end{array}$$

$$= f[0,0] \cdot \delta_2[n, m] + f[0,1] \cdot \delta_2[n, m-1] + \dots + f[2,2] \cdot \delta[n-2, n-2]$$

Linear Shift Invariant (LSI) systems

- 3 properties we need:

1. *We know what happens when we send a delta function through a LSI system:*

$$\delta_2[n, m] \rightarrow \boxed{\text{System } \mathcal{S}} \rightarrow h[n, m]$$

2. *We also know that LSI systems shift the output if the input is shifted:*

$$\delta_2[n - k, m - l] \rightarrow \boxed{\text{System } \mathcal{S}} \rightarrow h[n - k, m - l]$$

3. *Finally, the superposition principle:*

$$S[\alpha f_i[n, m] + \beta f_j[k, l]] = \alpha S[f_i[n, m]] + \beta S[f_j[k, l]]$$

Linear Shift Invariant (LSI) systems

- We have:

$$\begin{aligned} f[n, m] &= \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot \delta_2[n - k, m - l] \\ &\xrightarrow{S} \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot S\{\delta_2[n - k, m - l]\} \end{aligned}$$

- Recall, by the shift invariance property that:

$$S\{\delta_2[n - k, m - l]\} = h[n - k, m - l]$$

Linear Shift Invariant (LSI) systems

- We have:

$$\begin{aligned} f[n, m] &= \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot \delta_2[n - k, m - l] \\ &\xrightarrow{S} \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot S\{\delta_2[n - k, m - l]\} \end{aligned}$$

- Which means,

$$f[n, m] \xrightarrow{S} \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$

Linear Shift Invariant (LSI) systems

- An LSI system is completely specified by its impulse response.
 - For any input f , we can compute the output g in terms of the impulse response h .

$$f[n, m] \xrightarrow{S} g[n, m]$$

$$f[n, m] \xrightarrow{S} \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$

$$f[n, m] * h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$

Discrete Convolution

$$g[n, m] = f[n, m] * h[n, m]$$

Properties

- Associative property:

$$f * (g * h) = (f * g) * h$$

- Distributive property:

$$f * (g + h) = (f * g) + (f * h)$$

- The order doesn't matter! (Commutative)

$$g * h = h * g$$



Summary

- Systems and Filters
- LSI systems and convolution



What we will learn today?

- Color
- Image histograms
- Images as functions
- Linear systems (filters)
- Convolution and correlation



What we will learn today?

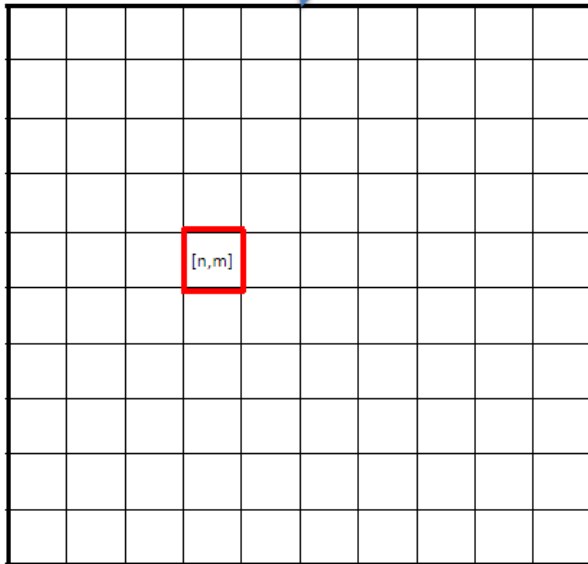
- **Convolution**
- **Correlation**

– *Some background reading:*

Forsyth and Ponce, Computer Vision, Chapter 7

2D Discrete Convolution

$$f[n, m] * h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$



Output $f * h$

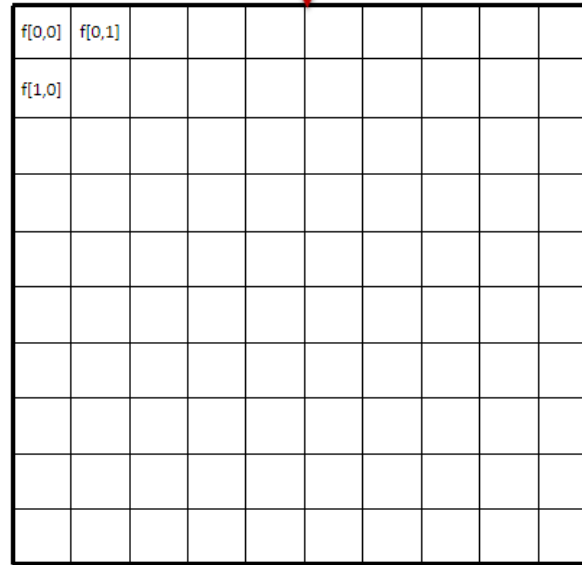


Image $f[k, l]$

$h[-1, -1]$	$h[-1, 0]$	$h[-1, 1]$
$h[0, -1]$	$h[0, 0]$	$h[0, 1]$
$h[1, -1]$	$h[1, 0]$	$h[1, 1]$

Kernel $h[k, l]$

2D Discrete Convolution

$$f[n, m] * h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$

$h[-1, -1]$	$h[-1, 0]$	$h[-1, 1]$
$h[0, -1]$	$h[0, 0]$	$h[0, 1]$
$h[1, -1]$	$h[1, 0]$	$h[1, 1]$

Kernel $h[k, l]$

Fold

$h[1, 1]$	$h[1, 0]$	$h[1, -1]$
$h[0, 1]$	$h[0, 0]$	$h[0, -1]$
$h[-1, 1]$	$h[-1, 0]$	$h[-1, -1]$

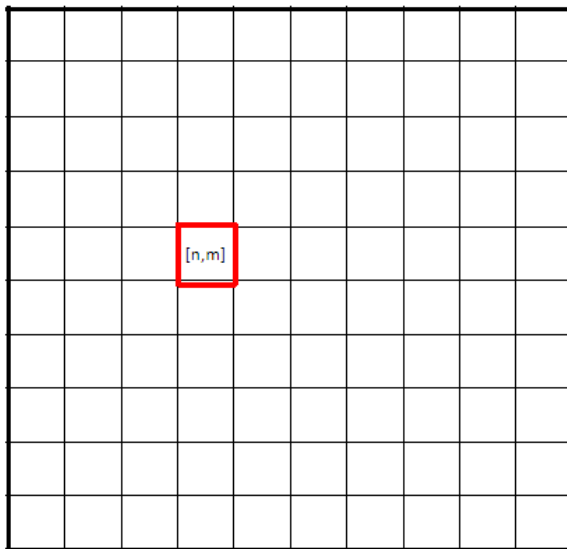
Kernel $h[-k, -l]$

Shift

Kernel $h[n-k, m-l]$

2D Discrete Convolution

$$f[n, m] * h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$



Output $f * h$

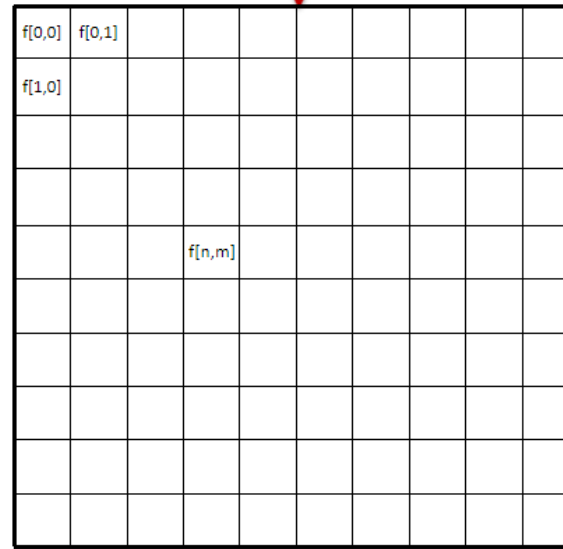
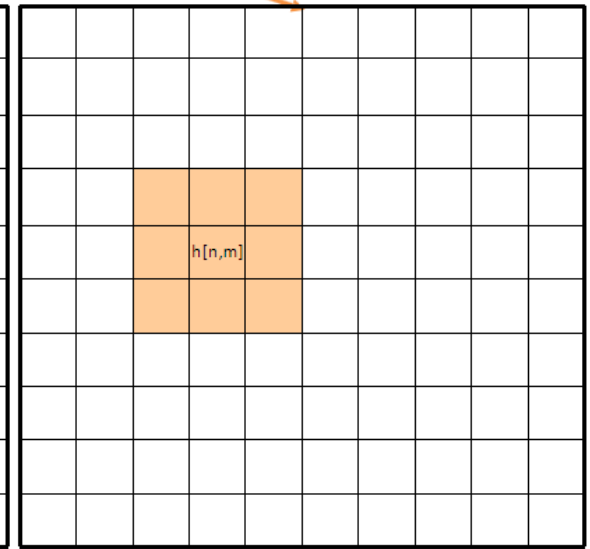


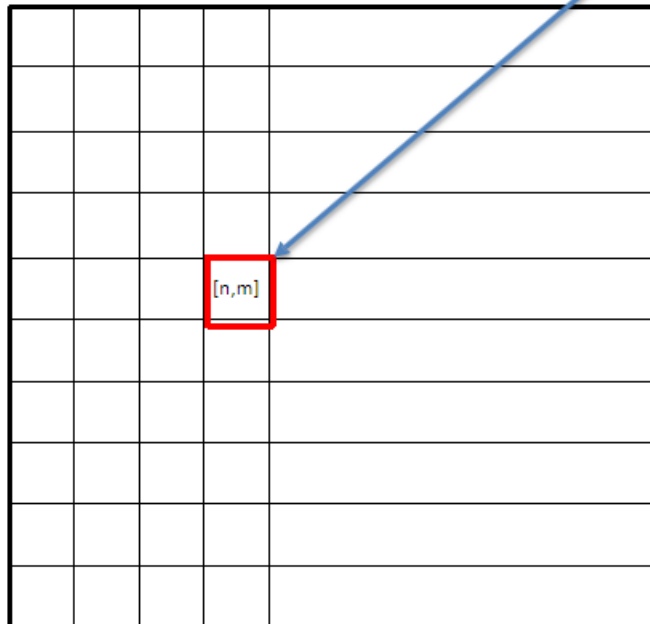
Image $f[k, l]$



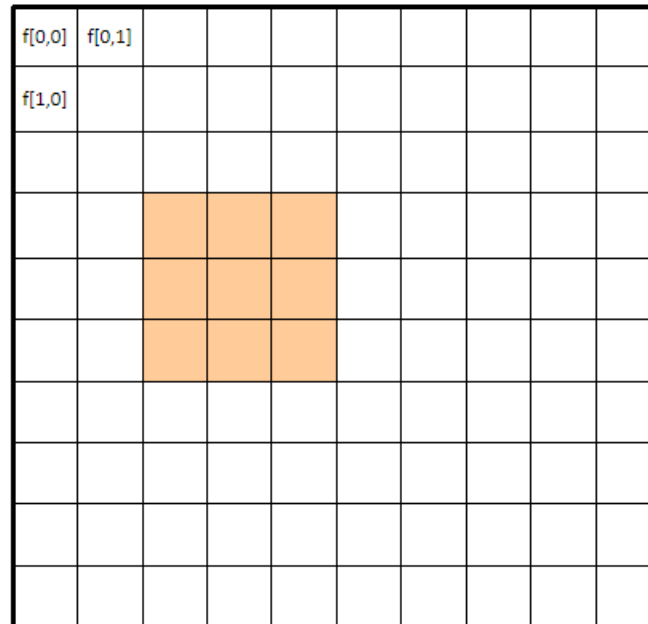
Kernel $h[n-k, m-l]$

2D Discrete Convolution

$$f[n, m] * h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$



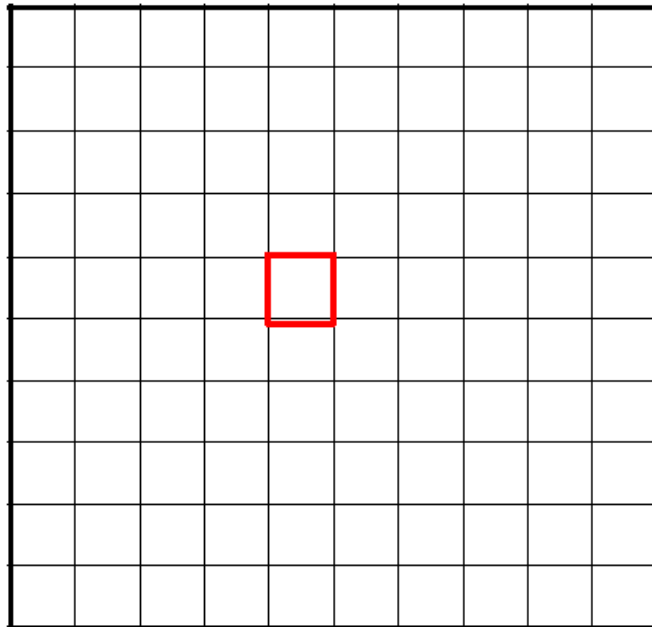
Output $f * h$



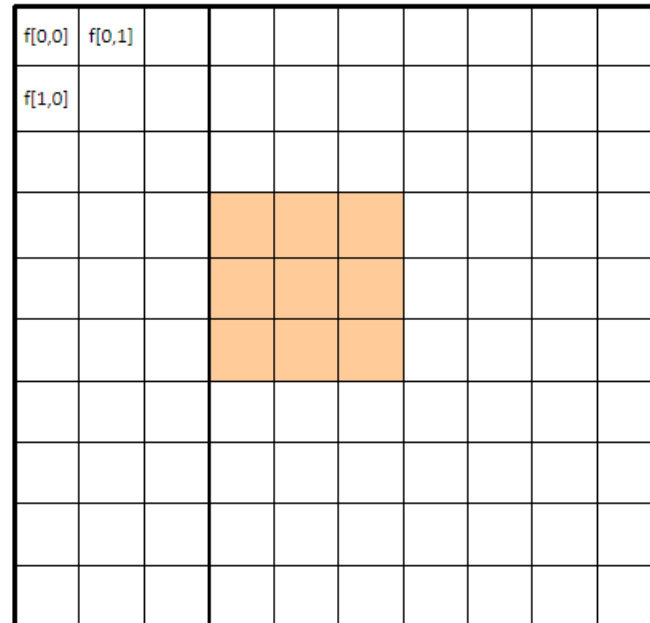
Element-wise multiplication
Image $f[k, l] \cdot$ Kernel $h[n-k, m-l]$

2D Discrete Convolution

$$f[n, m] * h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$



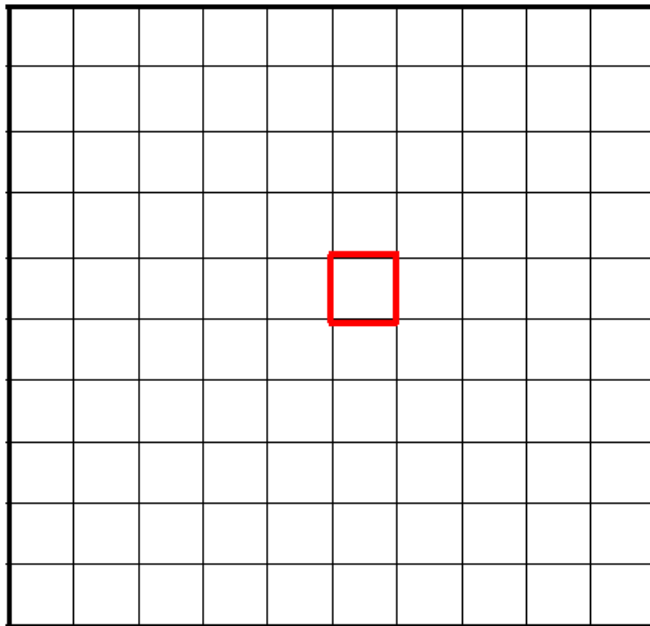
Output $f * h$



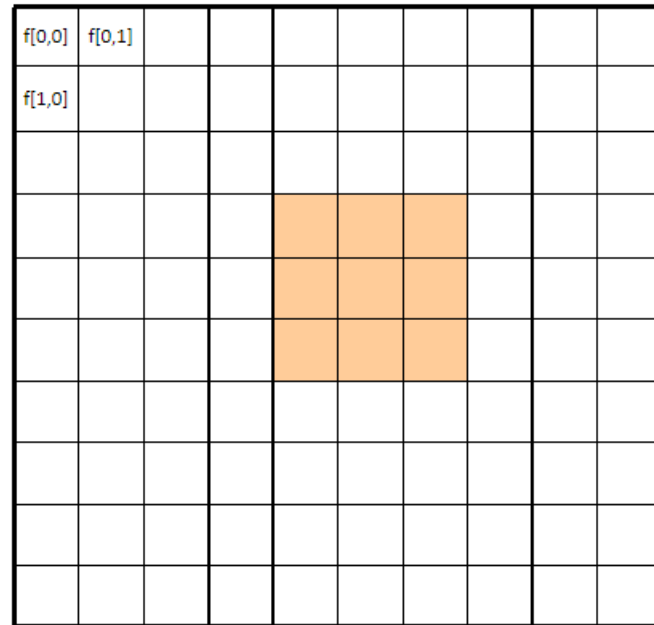
Element-wise multiplication
Image $f[k, l] \cdot$ Kernel $h[n-k, m-l]$

2D Discrete Convolution

$$f[n, m] * h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$



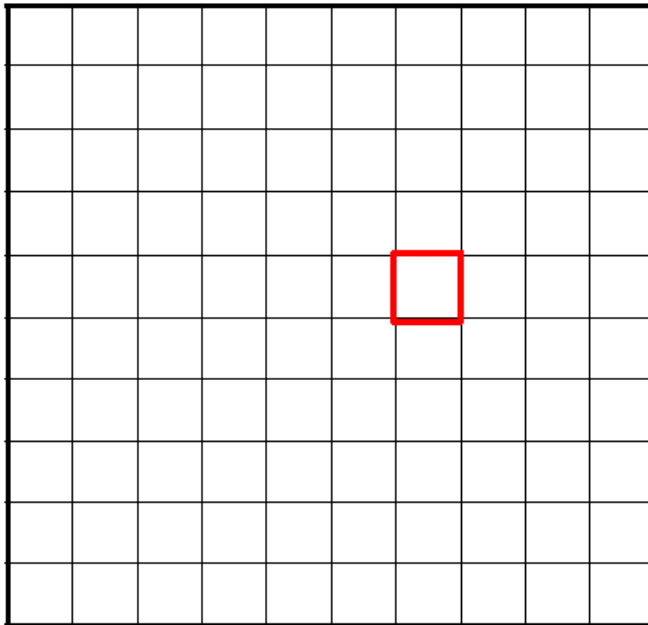
Output $f * h$



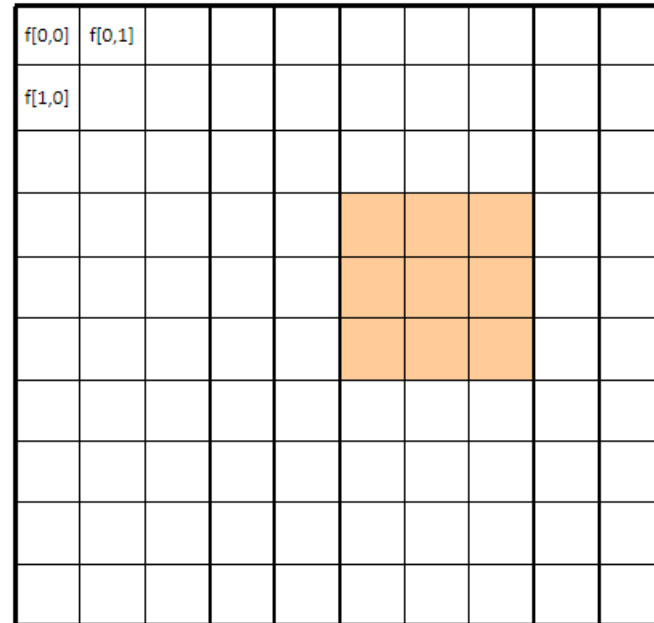
Element-wise multiplication
Image $f[k, l] \cdot$ Kernel $h[n-k, m-l]$

2D Discrete Convolution

$$f[n, m] * h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$



Output $f * h$



Element-wise multiplication
Image $f[k, l] \cdot$ Kernel $h[n-k, m-l]$

2D Discrete Convolution

$$f[n, m] * h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n - k, m - l]$$

- **Algorithm:**

1. *Fold $h[k, l]$ about origin to form $h[-k, -l]$*
2. *Shift the folded results by n, m to form $h[n - k, m - l]$*
3. *Multiply $h[n - k, m - l]$ by $f[k, l]$*
4. *Sum over all k, l*
5. *Repeat for every n, m*

2D convolution example

1	2	3
4	5	6
7	8	9

Input

		m		
n	-1	-1	-2	-1
	0	0	0	0
	1	1	2	1

Kernel

-13	-20	-17
-18	-24	-18
13	20	17

Output

2D convolution example

1	2	1	
0	0	0	
-1	-2	-1	

$$\begin{aligned}
 &= x[-1,-1] \cdot h[1,1] + x[0,-1] \cdot h[0,1] + x[1,-1] \cdot h[-1,1] \\
 &\quad + x[-1,0] \cdot h[1,0] + x[0,0] \cdot h[0,0] + x[1,0] \cdot h[-1,0] \\
 &\quad + x[-1,1] \cdot h[1,-1] + x[0,1] \cdot h[0,-1] + x[1,1] \cdot h[-1,-1] \\
 &= 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 1 + 0 \cdot 0 + 1 \cdot 0 + 2 \cdot 0 + 0 \cdot (-1) + 4 \cdot (-2) + 5 \cdot (-1) = -13
 \end{aligned}$$

-13	-20	-17
-18	-24	-18
13	20	17

Output

Slide credit: Song Ho Ahn

2D convolution example

1	2	1
0	0	0
1	2	3
-1	-2	-1
4	5	6
7	8	9

$$\begin{aligned} &= x[0,-1] \cdot h[1,1] + x[1,-1] \cdot h[0,1] + x[2,-1] \cdot h[-1,1] \\ &\quad + x[0,0] \cdot h[1,0] + x[1,0] \cdot h[0,0] + x[2,0] \cdot h[-1,0] \\ &\quad + x[0,1] \cdot h[1,-1] + x[1,1] \cdot h[0,-1] + x[2,1] \cdot h[-1,-1] \\ &= 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 1 + 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 0 + 4 \cdot (-1) + 5 \cdot (-2) + 6 \cdot (-1) = -20 \end{aligned}$$

-13	-20	-17
-18	-24	-18
13	20	17

Output

Slide credit: Song Ho Ahn

2D convolution example

		1	2	1	
1	0	2	0	0	
4	-1	5	-2	-1	
7	8	9			

$$\begin{aligned}
 &= x[1,-1] \cdot h[1,1] + x[2,-1] \cdot h[0,1] + x[3,-1] \cdot h[-1,1] \\
 &\quad + x[1,0] \cdot h[1,0] + x[2,0] \cdot h[0,0] + x[3,0] \cdot h[-1,0] \\
 &\quad + x[1,1] \cdot h[1,-1] + x[2,1] \cdot h[0,-1] + x[3,1] \cdot h[-1,-1] \\
 &= 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 1 + 2 \cdot 0 + 3 \cdot 0 + 0 \cdot 0 + 5 \cdot (-1) + 6 \cdot (-2) + 0 \cdot (-1) = -17
 \end{aligned}$$

-13	-20	-17
-18	-24	-18
13	20	17

Output

Slide credit: Song Ho Ahn

2D convolution example

1	2	1	
	1	2	3
0	0	0	
	4	5	6
-1	-2	-1	
	7	8	9

$$\begin{aligned}
 &= x[-1,0] \cdot h[1,1] + x[0,0] \cdot h[0,1] + x[1,0] \cdot h[-1,1] \\
 &\quad + x[-1,1] \cdot h[1,0] + x[0,1] \cdot h[0,0] + x[1,1] \cdot h[-1,0] \\
 &\quad + x[-1,2] \cdot h[1,-1] + x[0,2] \cdot h[0,-1] + x[1,2] \cdot h[-1,-1] \\
 &= 0 \cdot 1 + 1 \cdot 2 + 2 \cdot 1 + 0 \cdot 0 + 4 \cdot 0 + 5 \cdot 0 + 0 \cdot (-1) + 7 \cdot (-2) + 8 \cdot (-1) = -18
 \end{aligned}$$

-13	-20	-17
-18	-24	-18
13	20	17

Output

Slide credit: Song Ho Ahn

2D convolution example

1 1	2 2	1 3
0 4	0 5	0 6
-1 7	-2 8	-1 9

$$\begin{aligned} &= x[0,0] \cdot h[1,1] + x[1,0] \cdot h[0,1] + x[2,0] \cdot h[-1,1] \\ &\quad + x[0,1] \cdot h[1,0] + x[1,1] \cdot h[0,0] + x[2,1] \cdot h[-1,0] \\ &\quad + x[0,2] \cdot h[1,-1] + x[1,2] \cdot h[0,-1] + x[2,2] \cdot h[-1,-1] \\ &= 1 \cdot 1 + 2 \cdot 2 + 3 \cdot 1 + 4 \cdot 0 + 5 \cdot 0 + 6 \cdot 0 + 7 \cdot (-1) + 8 \cdot (-2) + 9 \cdot (-1) = -24 \end{aligned}$$

-13	-20	-17
-18	-24	-18
13	20	17

Output

Slide credit: Song Ho Ahn

2D convolution example

1	1	2	3	1
4	0	5	0	0
7	-1	8	-2	9

$$\begin{aligned}
 &= x[1,0] \cdot h[1,1] + x[2,0] \cdot h[0,1] + x[3,0] \cdot h[-1,1] \\
 &\quad + x[1,1] \cdot h[1,0] + x[2,1] \cdot h[0,0] + x[3,1] \cdot h[-1,0] \\
 &\quad + x[1,2] \cdot h[1,-1] + x[2,2] \cdot h[0,-1] + x[3,2] \cdot h[-1,-1] \\
 &= 2 \cdot 1 + 3 \cdot 2 + 0 \cdot 1 + 5 \cdot 0 + 6 \cdot 0 + 0 \cdot 0 + 8 \cdot (-1) + 9 \cdot (-2) + 0 \cdot (-1) = -18
 \end{aligned}$$

-13	-20	-17
-18	-24	-18
13	20	17

Output

Slide credit: Song Ho Ahn

Convolution in 2D - examples



Original



•0	•0	•0
•0	•1	•0
•0	•0	•0

=

?

Convolution in 2D - examples



Original



•0	•0	•0
•0	•1	•0
•0	•0	•0



Filtered
(no change)

Convolution in 2D - examples



Original

*

•0	•0	•0
•0	•0	•1
•0	•0	•0

=

?

Convolution in 2D - examples



Original



•0	•0	•0
•0	•0	•1
•0	•0	•0



Shifted right
By 1 pixel

Convolution in 2D - examples



Original

$$\ast \frac{1}{9} \begin{bmatrix} \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \end{bmatrix} = ?$$

Convolution in 2D - examples



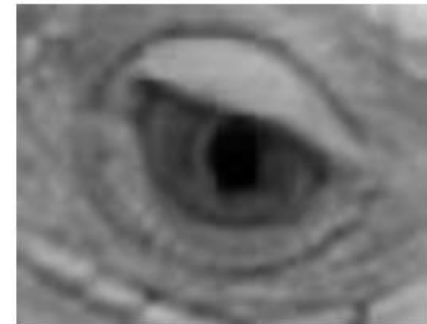
Original

*

$\frac{1}{9}$

•1	•1	•1
•1	•1	•1
•1	•1	•1

=



Blur (with a
box filter)

Convolution in 2D - examples



Original

$$* \left(\begin{bmatrix} \bullet 0 & \bullet 0 & \bullet 0 \\ \bullet 0 & \bullet 2 & \bullet 0 \\ \bullet 0 & \bullet 0 & \bullet 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \end{bmatrix} \right) = ?$$

(Note that filter sums to 1)

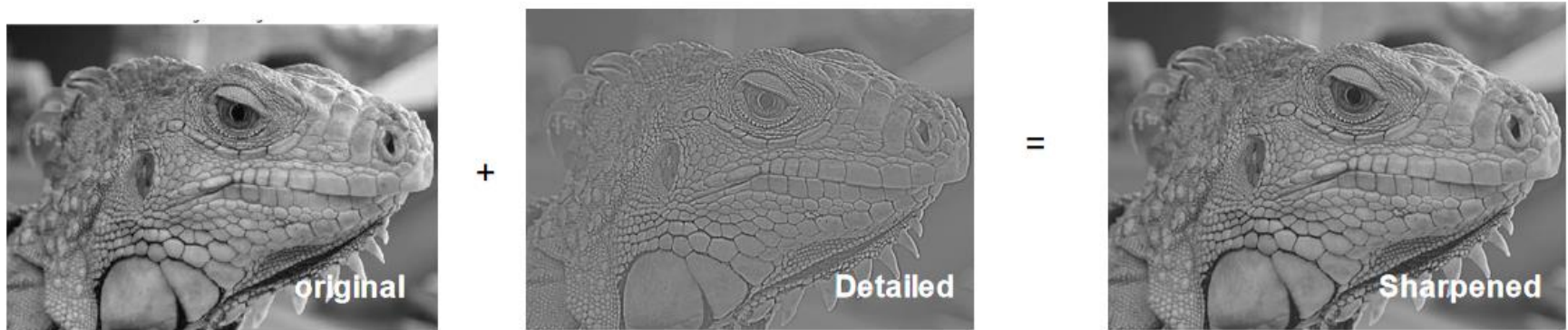
“details of the image”

$$\begin{bmatrix} \bullet 0 & \bullet 0 & \bullet 0 \\ \bullet 0 & \bullet 1 & \bullet 0 \\ \bullet 0 & \bullet 0 & \bullet 0 \end{bmatrix} + \begin{bmatrix} \bullet 0 & \bullet 0 & \bullet 0 \\ \bullet 0 & \bullet 1 & \bullet 0 \\ \bullet 0 & \bullet 0 & \bullet 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \end{bmatrix}$$

What does blurring take away?



- Let's add it back:



Convolution in 2D - examples



Original

$$* \left(\begin{bmatrix} \bullet 0 & \bullet 0 & \bullet 0 \\ \bullet 0 & \bullet 2 & \bullet 0 \\ \bullet 0 & \bullet 0 & \bullet 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \end{bmatrix} \right) =$$

(Note that filter sums to 1)



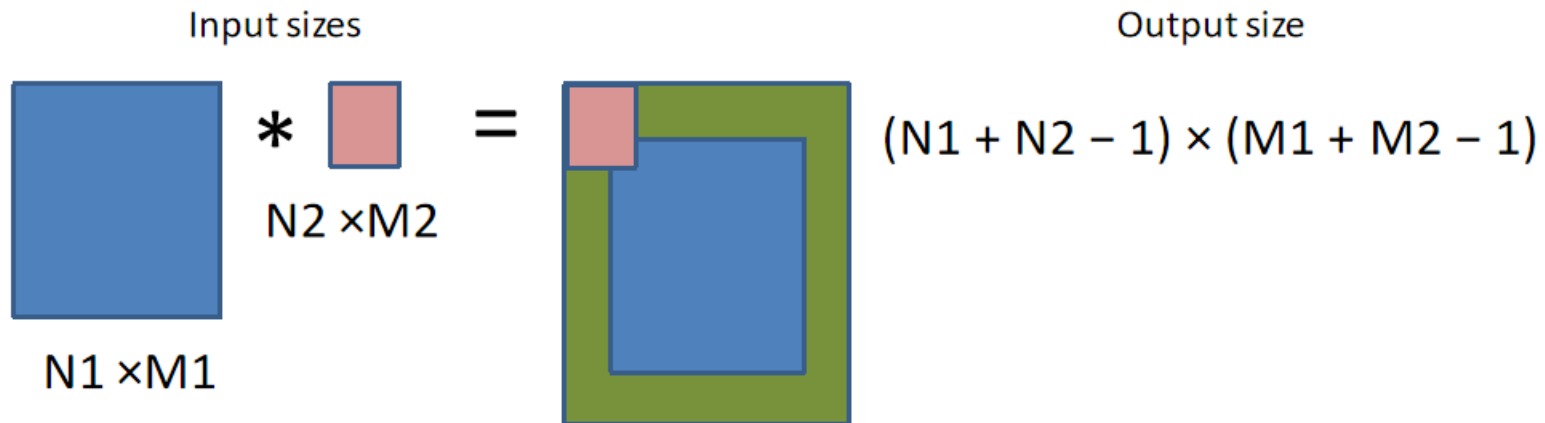
“details of the image”

$$\begin{bmatrix} \bullet 0 & \bullet 0 & \bullet 0 \\ \bullet 0 & \bullet 1 & \bullet 0 \\ \bullet 0 & \bullet 0 & \bullet 0 \end{bmatrix} + \begin{bmatrix} \bullet 0 & \bullet 0 & \bullet 0 \\ \bullet 0 & \bullet 1 & \bullet 0 \\ \bullet 0 & \bullet 0 & \bullet 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \\ \bullet 1 & \bullet 1 & \bullet 1 \end{bmatrix}$$

Sharpening filter: Accentuates differences with local average

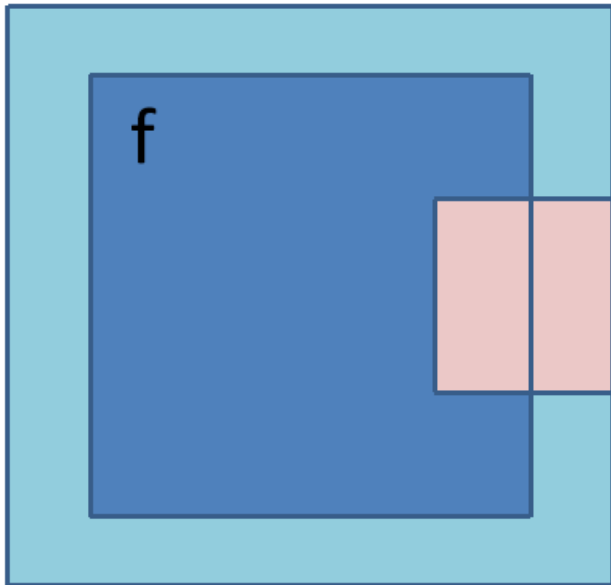
Implementation detail: Image support and edge effect

- A computer will only convolve **finite support signals**.
 - That is: images that are zero for n, m outside some rectangular region;
- Numpy's convolution performs 2D Discrete convolution of finite support signals.



Implementation detail: Image support and edge effect

- *A computer will only convolve finite support signals.*
- *What happens at the edge?*



h

- zero “padding”
- edge value replication
- mirror extension
- more (beyond the scope of this class)



What we will learn today?

- *Convolution*
- **Correlation**

– *Some background reading:*
Forsyth and Ponce, Computer Vision, Chapter 7

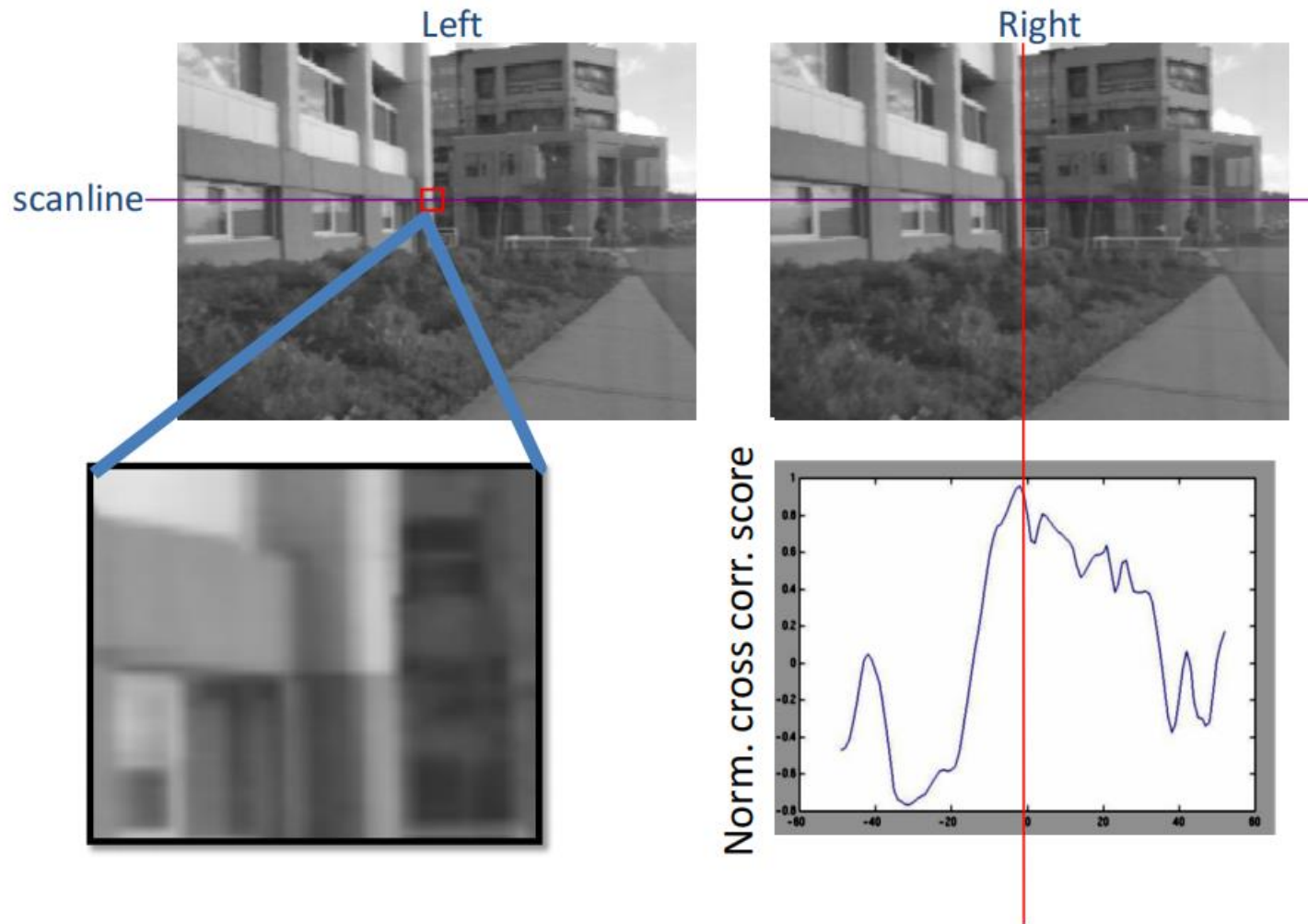
(Cross) correlation – symbol: **

- *Cross correlation of two 2D signals $f[n,m]$ and $h[n,m]$*

$$f[n, m] ** h[n, m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \cdot h[n + k, m + l]$$

- *Equivalent to a convolution without the flip*
- *Use it to measure ‘similarity’ between f and h .*

(Cross) correlation – example



(Cross) correlation – example



Cross Correlation Application: Vision system for TV remote control

- uses template matching

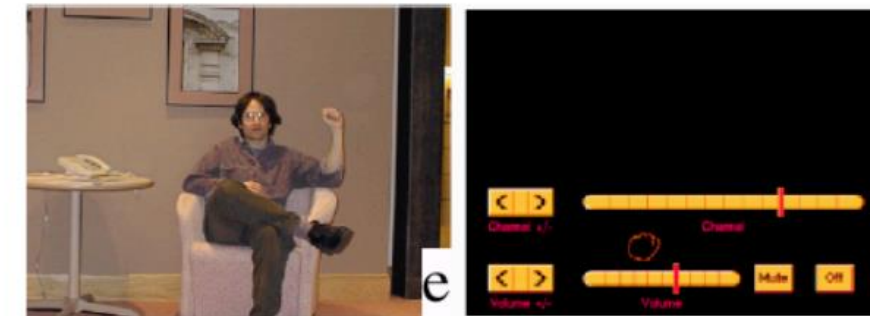
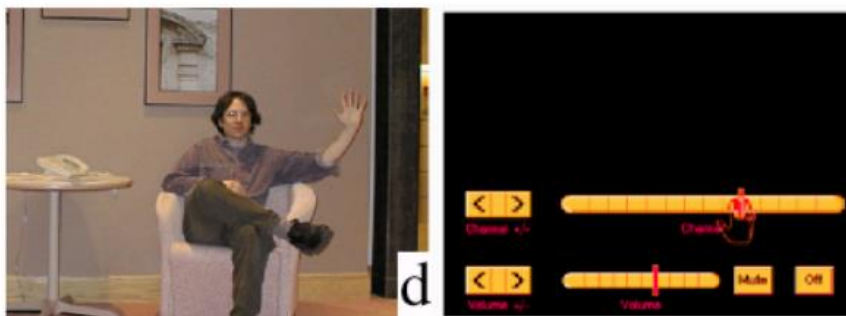
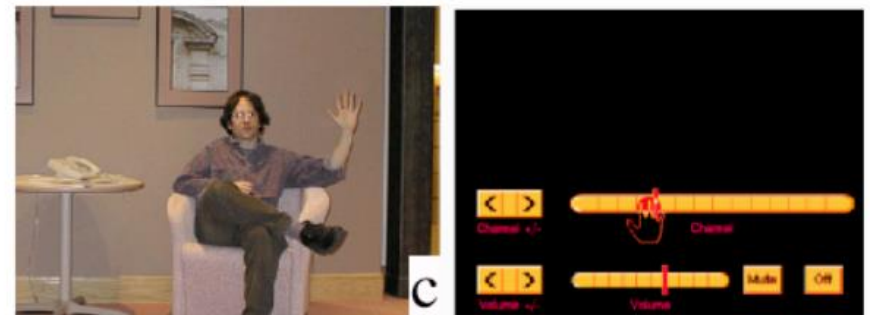
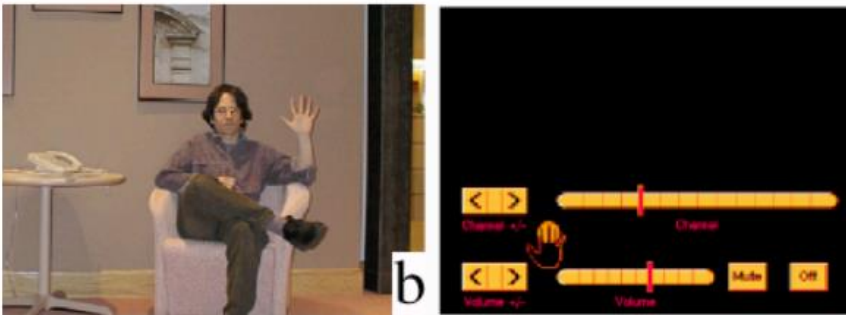
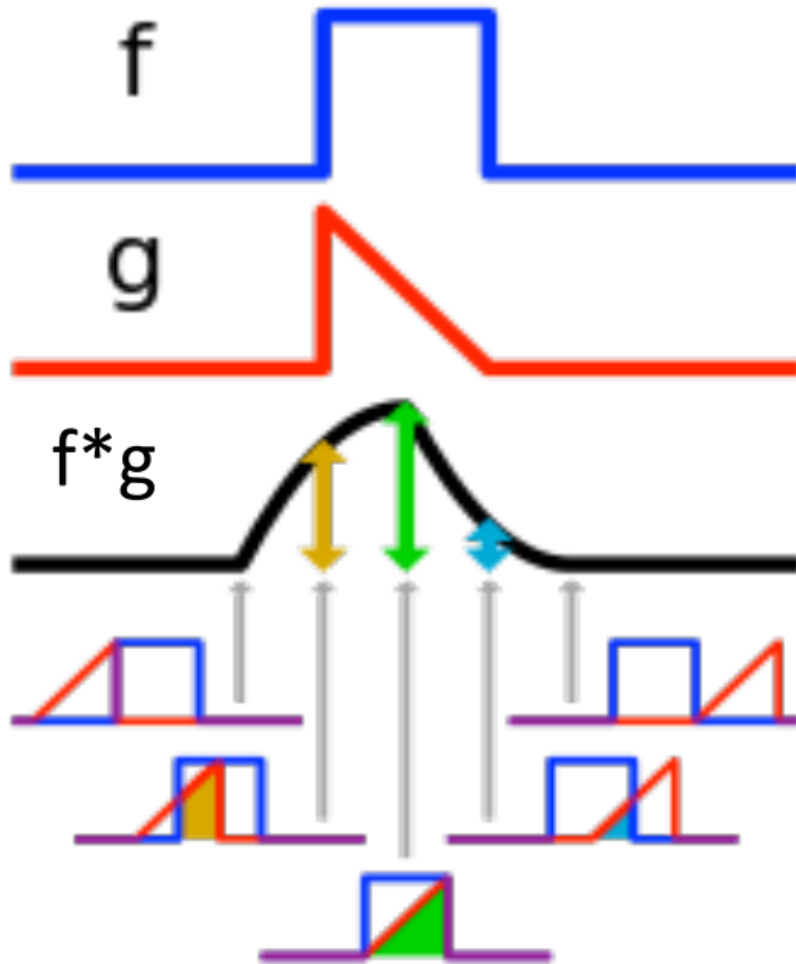


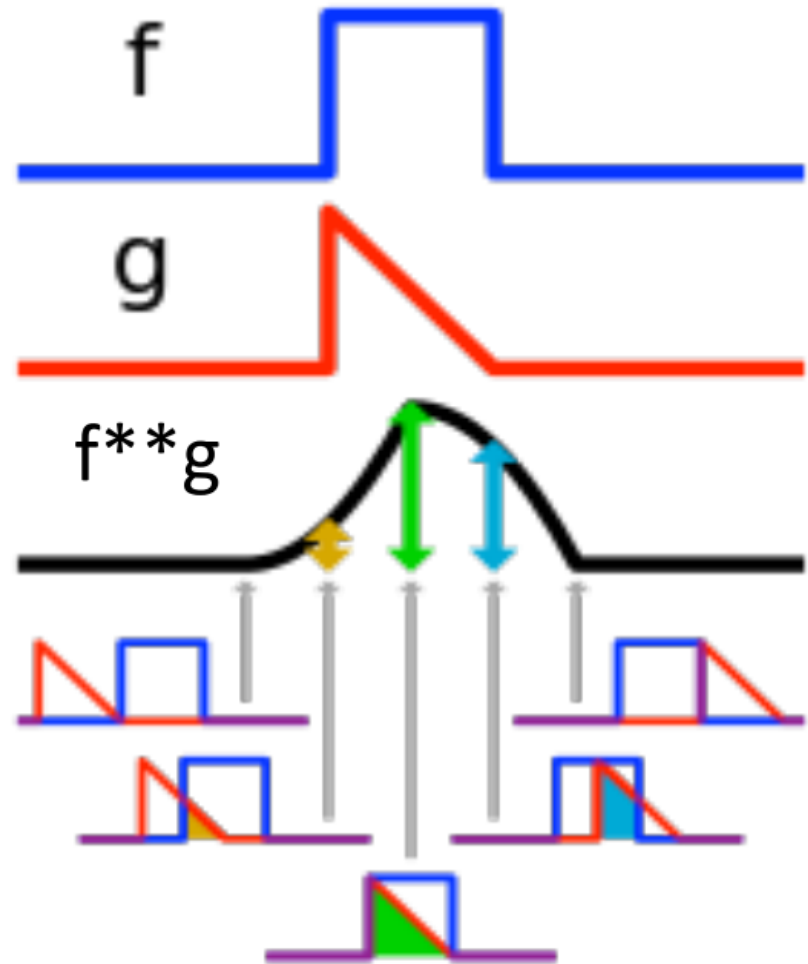
Figure from "Computer Vision for Interactive Computer Graphics," W.Freeman et al, IEEE Computer Graphics and Applications, 1998 copyright 1998, IEEE

(Cross) correlation – example

Convolution



Cross-correlation



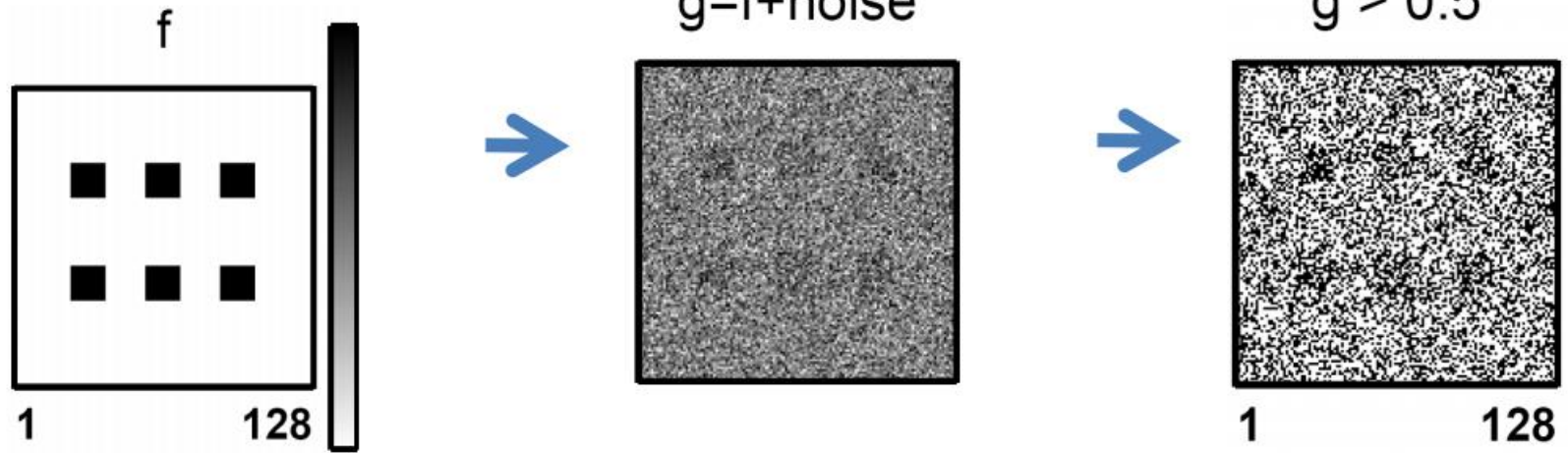
Properties

- Superposition (Linear)?
- Shift-invariant?
- Commutative?

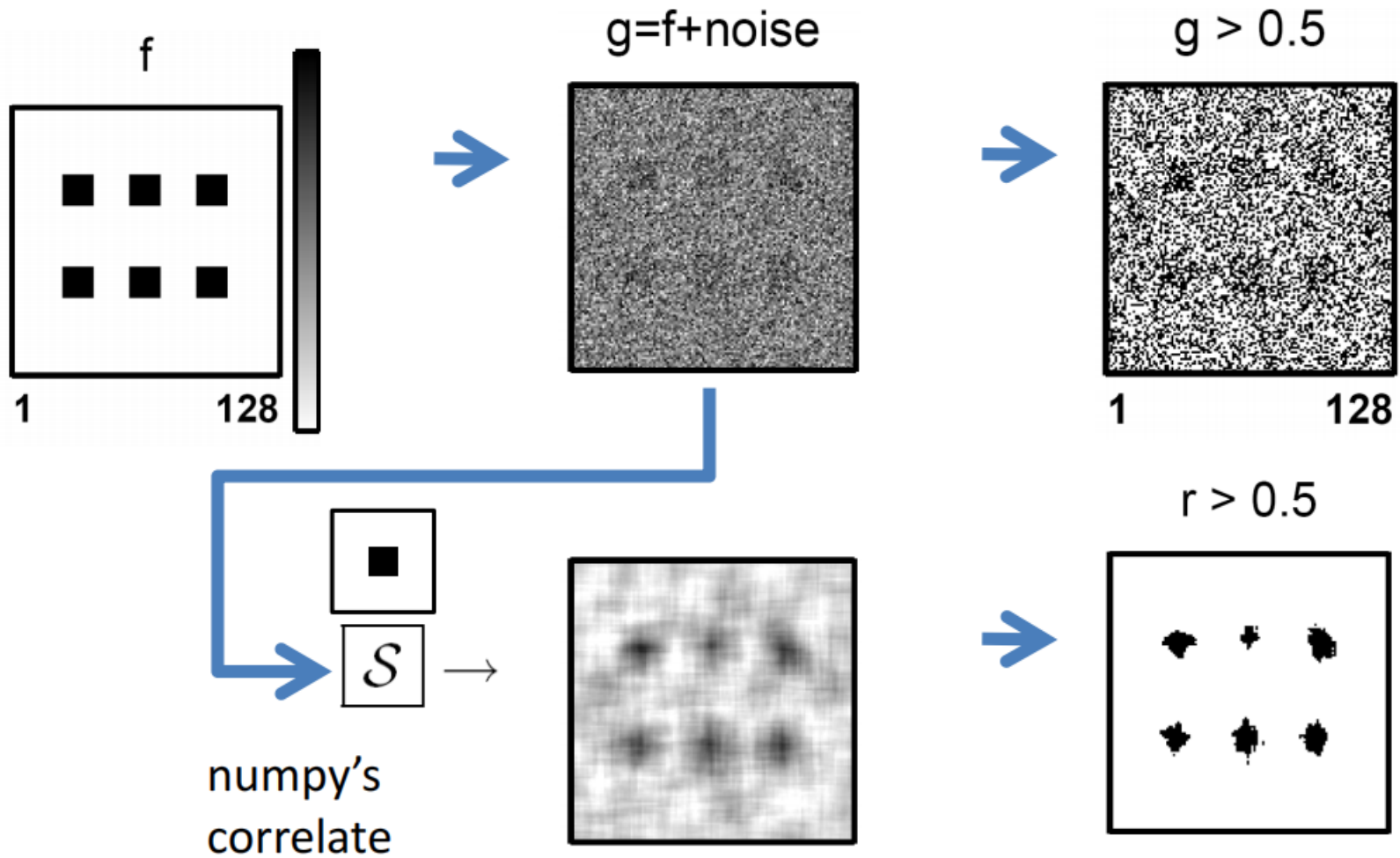
Convolution vs. (Cross) Correlation

- When is correlation equivalent to convolution?
- In other words, when is $f ** g = f * g$?

(Cross) correlation – example



(Cross) correlation – example



Courtesy of J. Fessler

Convolution vs. (Cross) Correlation

- A **convolution** is an integral that expresses the amount of overlap of one function as it is shifted over another function.
 - convolution is a filtering operation
- **Correlation** compares the **similarity** of **two** sets of **data**. Correlation computes a measure of similarity of two input signals as they are shifted by one another. The correlation result reaches a maximum at the time when the two signals match best .
 - correlation is a measure of relatedness of two signals



Summary

- *Convolution*
- *Correlation*